# 数据结构与算法

(Python语言描述 笔记)

(完结时间：2021/3/12 00:30:00)

(by 孟智超)

# 1 目录

1 目录
2 一、 引入概念
2.1 第一次尝试
2.2 算法的提出
2.3 第二次尝试
2.4 算法效率衡量
2.5 常见时间复杂度
2.6 Pyhton内置性能分析
2.7 数据结构
3 二、顺序表（线性表）
3.1 顺序表的形式
3.2 顺序表的结构与实现
3.3 顺序表的操作
3.4 Python中的顺序表
4 三、链表（线性表）
4.1 单向链表
4.2 链表与顺序表的对比
4.3 单向循环链表
4.4 双向链表
5 四、栈
5.1 栈结构实现
6 五、队列
6.1 队列的实现
6.2 双端队列
7 六、排序与搜索
7.1 冒泡排序
7.2 选择排序
7.3 插入排序
7.4 希尔排序
7.5 快速排序（必须要掌握）
7.6 归并排序
7.7 常见排序算法效率比较
7.8 搜索
8 七、树与树算法
8.1 二叉树
8.2 二叉树的遍历（BFS and DFS）

# 2 一、 引入概念

- 先看一道 题
  如果a+b+c=1000,且$a^2+b^2=c^2$，求所有组合的结果

## 2.1 第一次尝试

-

```
import time
from tqdm import tqdm

start_time = time.time()
for a in tqdm(range(0, 1001)):
    for b in range(0, 1001):
        for c in range(0, 1001):
            if (a + b + c == 1000) and (a ** 2 + b ** 2 == c ** 2):
                print(a, b, c)
end_time = time.time()
print("运行时间:", (end_time - start_time))
"""时间复杂度约为O(N^3)"""

# 运行结果:

  0%|          | 0/1001 [00:00<?, ?it/s]0 500 500
 20%|▉         | 200/1001 [00:54<03:00,  4.43it/s]200 375 425
 37%|▉▉▉       | 375/1001 [01:44<03:08,  3.32it/s]375 200 425
 50%|▉▉▉▉▉     | 499/1001 [02:21<02:52,  2.92it/s]500 0 500
100%|▉▉▉▉▉▉▉▉▉▉| 1001/1001 [04:33<00:00,  3.66it/s]
运行时间: 273.4053370952606
```

## 2.2 算法的提出

- 算法五大特性
  1. 输入： 具有0个或多个输入
  2. 输出： 算法至少有1个或多个输出
  3. 有穷性： 在有限步骤之后会自动而不会无限循环，且每个步骤可以在可接受时间内完成
  4. 确定性： 算法每一步都有确定的含义，不会出现二义性
  5. 可行性： 算法的每一步都是可行的，也就是说每一步都能够执行有限的次数完成

## 2.3 第二次尝试

-

```python
import time
from tqdm import tqdm

start_time = time.time()
for a in tqdm(range(0, 1001)):
    for b in range(0, 1001):
        c = 1000-a-b
        if (a + b + c == 1000) and (a ** 2 + b ** 2 == c ** 2):
            print(a, b, c)
end_time = time.time()
print("运行时间:", (end_time - start_time))
"""时间复杂度约为O(N^2)"""

# 运行结果
  0%|          | 0/1001 [00:00<?, ?it/s]0 500 500
 14%|▌         | 143/1001 [00:00<00:01, 701.95it/s]200 375 425
 33%|▌▌▌       | 335/1001 [00:00<00:01, 653.86it/s]375 200 425
 47%|▌▌▌▌      | 470/1001 [00:00<00:00, 661.20it/s]500 0 500
100%|▌▌▌▌▌▌▌▌▌▌| 1001/1001 [00:01<00:00, 674.74it/s]
运行时间: 1.4913501739501953
```

## 2.4 算法效率衡量

- 由于机器性能的原因，每台机器执行的总时间不同，但是执行的基本运算数量（基本运算步骤）大体相同
- 大O表示法 （注意渐近函数）
  对于单调的整数函数f，如果存在一个整数函数g和实常数c>0，使得对于充分大的n总有f(n)<=c*g(n)，就说函数g是f的一个 渐近函数 （忽略常数），记为f(n)=O(g(n))。也就是说，在趋向无穷的极限意义下，函数f的增长速度受到函数g的约束，亦即函数f与函数g的特征相似。
- 时间复杂度
  假设存在函数g，使得算法A处理规模为n的问题示例所用时间为T(n)=O(g(n))，则称O(g(n))为算法A的渐近时间复杂度，简称时间复杂度，记为T(n)
- 最坏时间复杂度 （我们通常说的都是最坏时间复杂度）
  - 算法完成工作最少需要多少基本操作，即**最优时间复杂度**（价值不大）
  - 算法完成工作最多需要多少基本操作，即**最坏时间复杂度**
  - 算法完成工作平均需要多少基本操作，即**平均时间复杂度**

    对于最坏时间复杂度，提供了一种保证，表明算法在此种程度的基本操作中一定能完成工作。

对于平均时间复杂度，是对算法的一个全面评价，因此它完整全面的反映了这个算法的性质。但另一方面，这种衡量并没有保证，不是每个计算都能在这个基本操作内完成。而且，对于平均情况的计算，也会因为应用算法的实例分布可能并不均匀而难以计算。

**因此，我们主要关注算法的最坏情况，亦即最坏时间复杂度。**

- 时间复杂度 `计算` 基本规则
    1. 基本操作，即只有常数项，认为其时间复杂度为O(1)
    2. `顺序` 结构，时间复杂度按**加法**进行计算
    3. `循环` 结构，时间复杂度按**乘法**进行计算
    4. `分支` 结构，时间复杂度**取最大值**
- 判断一个算法的效率时，往往只需要关注操作数量的最高次项，其它次要项和常数项可以忽略在没有特殊说明时，我们所分析的算法的时间复杂度都是指**最坏时间复杂度**

---

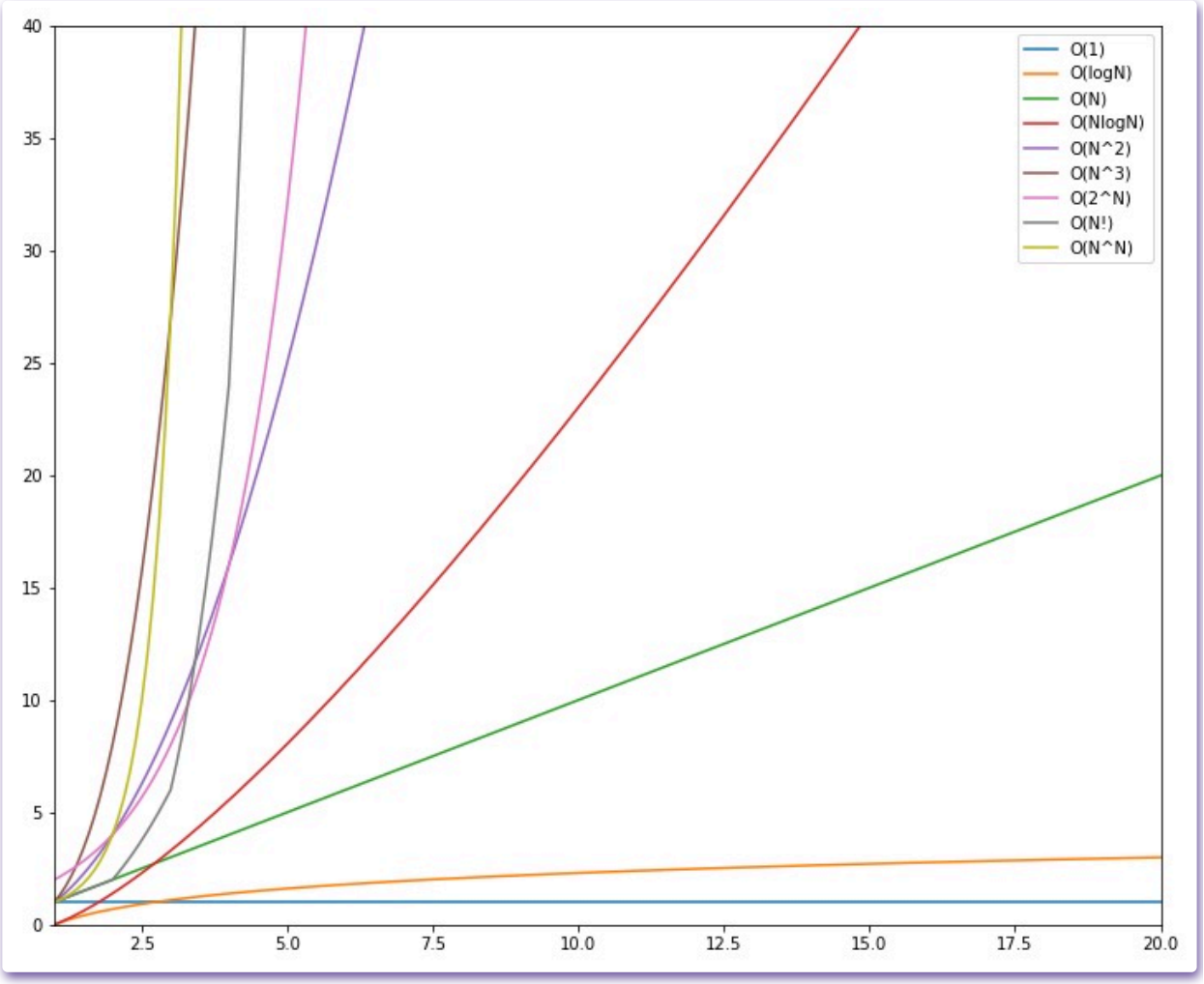
## 2.5 常见时间复杂度

- 常见时间复杂度

| 执行次数函数举例 | 阶 | 非正式术语 |
|---|---|---|
| 12 | O(1) | 常数阶 |
| $5\log_2 n + 20$ | O(logn) | 对数阶 |
| 2n+3 | O(n) | 线性阶 |
| $2n + 3n\log_2 n + 19$ | O(nlogn) | 对数线性阶 |
| $3n^2 + 2n + 1$ | O($n^2$) | 平方阶 |
| $6n^3 + 2n^2 + 3n + 4$ | O($n^3$) | 立方阶 |
| $2^n$ | O($2^n$) | 指数阶 |

- 所消耗的时间从小到大

  O(1) < O(log N) < O(N) < O(N log N) < O($N^2$) < O($N^3$) < O($2^N$) < O(N!) < O($N^N$)




## 2.6 Pyhton内置性能分析

- timeit模块

  `timeit` 模块可以用来测试一小段Python代码的执行速度。

  `class timeit.Timer(stmt='pass', setup='pass', timer=<timer function>)`

  `Timer` 是测量小段代码执行速度的类。

  `stmt` 参数是要测试的代码语句（`statment`）；

  `setup` 参数是运行代码时需要的设置；

  `timer` 参数是一个定时器函数，与平台有关。

  `timeit.Timer.timeit(number=1000000)`

  Timer类中测试语句执行速度的对象方法。`number` 参数是测试代码时的测试次数，默认为1000000次。方法返回执行代码的平均耗时，一个 `float` 类型的秒数。

- list的操作测试

```
from timeit import Timer

```

```python
def t1():
    li1 = []
    for i in range(10000):
        li1.append(i)

def t2():
    li2 = []
    for i in range(10000):
        li2 += [i]

def t3():
    li3 = []
    for i in range(10000):
        li3 = li3 + [i]

def t4():
    li4 = [i for i in range(10000)]

def t5():
    li5 = list(range(10000))

def t6():
    li6 = []
    for i in range(10000):
        li6.extend([i])

timer1 = Timer('t1()', "from __main__ import t1")
print("append 运行时间:", timer1.timeit(number=1000), 'seconds')
timer2 = Timer('t2()', "from __main__ import t2")
print("+= 运行时间:", timer2.timeit(number=1000), 'seconds')
timer3 = Timer('t3()', "from __main__ import t3")
print("a=a+[i] 运行时间:", timer3.timeit(number=1000), 'seconds')
timer4 = Timer('t4()', "from __main__ import t4")
print("[i for i in range] 运行时间:", timer4.timeit(number=1000), 'seconds')
timer5 = Timer('t5()', "from __main__ import t5")
print("list(range()) 运行时间:", timer5.timeit(number=1000), 'seconds')
timer6 = Timer('t6()', "from __main__ import t6")
print("extend 运行时间:", timer6.timeit(number=1000), 'seconds')

#运行结果
append 运行时间: 1.12506746 seconds
+= 运行时间: 1.2395838000000001 seconds
a=a+[i] 运行时间: 228.37068485 seconds
[i for i in range] 运行时间: 0.6368972020000001 seconds
list(range()) 运行时间: 0.45499822300001824 seconds
extend 运行时间: 2.2393252930000074 seconds

"""
特别需要注意的是:
+= 类似于 extend, 如lia = lia + [i] 是直接对 列表 lia 进行操作
```

```
lia = lia + [i] 是 类似于将两个列表 lia 和 [i] 相加再生成lia
所以 extend和+= 时间差不多，而 = + 时间很长，要注意
"""
```

- `append 和 insert`

```python
from timeit import Timer

def t1():
    li = []
    for i in range(10000):
        li.append(i)

def t2():
    li = []
    for i in range(10000):
        li.insert(0, i)

timer1 = Timer('t1()', "from __main__ import t1")
print('append 运行时间:', timer1.timeit(number=1000), "seconds")
timer2 = Timer('t2()', "from __main__ import t2")
print('insert 运行时间:', timer2.timeit(number=1000), "seconds")

#运行结果
append 运行时间: 1.075284503
insert 运行时间: 40.553926374
```

测试 `append` 和 `insert` 操作：从结果可以看出， `append` 效率高于 `insert` 。

- `pop操作测试`

```python
from timeit import Timer

x_1 = list(range(2000000))
pop_zero = Timer('x_1.pop(0)', 'from __main__ import x_1')
print('pop_zero:', pop_zero.timeit(number=1000), "seconds")

x_2 = list(range(2000000))
pop_end = Timer('x_2.pop()', 'from __main__ import x_2')
print('pop_end: %.10f' % pop_end.timeit(number=1000), "seconds")

# 运行结果
pop_zero: 1.952334484 seconds
pop_end: 0.0001876510 seconds
```

测试pop操作：从结果可以看出，pop最后一个元素的效率远远高于pop第一个元素

- `list内置操作时间复杂度`

| Operation | Big-O Efficiency |
|---|---|
| indexx[] | $O(1)$ |
| index assignment | $O(1)$ |
| append | $O(1)$ |
| pop() | $O(1)$ |
| pop(i) | $O(n)$ |
| insert(i,item) | $O(n)$ |
| del operator | $O(n)$ |
| iteration | $O(n)$ |
| contains (in) | $O(n)$ |
| get slice [x:y] | $O(k)$ |
| del slice | $O(n)$ |
| set slice | $O(n+k)$ |
| reverse | $O(n)$ |
| concatenate | $O(k)$ |
| sort | $O(n \log n)$ |
| multiply | $O(nk)$ |

Table 2.2: Big-O Efficiency of Python List Operators

- dict内置操作时间复杂度

| Operation | Big-O Efficiency |
|---|---|
| copy | $O(n)$ |
| get item | $O(1)$ |
| set item | $O(1)$ |
| delete item | $O(1)$ |
| contains (in) | $O(1)$ |
| iteration | $O(n)$ |

Table 2.3: Big-O Efficiency of Python Dictionary Operations

---

## 2.7 数据结构

- 数据结构

```
我们如何用Python中的类型来保存一个班的学生信息？ 如果想要快速的通过学生姓名获取其信息呢?
```

实际上当我们在思考这个问题的时候，我们已经用到了数据结构。列表和字典都可以存储

一个班的学生信息，但是想要在列表中获取一名同学的信息时，就要遍历这个列表，其时间复杂度为O(n)，而使用字典存储时，可将学生姓名作为字典的键，学生信息作为值，进而查询时不需要遍历便可快速获取到学生信息，其时间复杂度为O(1)。

我们为了解决问题，需要将数据保存下来，然后根据数据的存储方式来设计算法实现进行处理，那么数据的存储方式不同就会导致需要不同的算法进行处理。我们希望算法解决问题的**效率越快越好**，于是我们就需要考虑数据究竟如何保存的问题，这就是数据结构。

在上面的问题中我们可以选择Python中的列表或字典来存储学生信息。列表和字典就是Python内建帮我们封装好的两种数据结构。

- 概念

  **数据**是一个抽象的概念，将其进行分类后得到程序设计语言中的基本类型。如：int，float，char等。数据元素之间不是独立的，存在特定的关系，这些关系便是结构。数据结构指数据对象中数据元素之间的关系。

  Python给我们提供了很多现成的数据结构类型，这些系统自己定义好的，不需要我们自己去定义的数据结构叫做Python的内置数据结构，比如列表、元组、字典。而有些数据组织方式，Python系统里面没有直接定义，需要我们自己去定义实现这些数据的组织方式，这些数据组织方式称之为Python的扩展数据结构，比如栈，队列等。

- 算法与数据结构的区别

  数据结果只是静态的描述了数据元素之间的关系。

  高效的程序需要在数据结构的基础上设计和选择算法。

  **程序 = 数据结构 + 算法**

  **总结：算法是为了解决实际问题而设计的，数据结构是算法需要处理的问题载体**

- 抽象数据类型(Abstract Data Type)

  抽象数据类型(ADT)的含义是指一个数学模型以及定义在此数学模型上的一组操作。即把数据类型和数据类型上的运算捆在一起，进行封装。引入抽象数据类型的目的是把数据类型的表示和数据类型上运算的实现与这些数据类型和运算在程序中的引用隔开，使它们相互独立。

  最常见的数据运算有 5种

  1. 插入
  2. 删除
  3. 修改
  4. 查找
  5. 排序

---

# 3 二、顺序表（线性表）

- 在程序中，经常需要将一组（通常是同为某个类型的）数据元素作为整体管理和使用，需要创建这种元素组，用变量记录它们，传进传出函数等。一组数据中包含的元素个数可能发生变化（可以增加或删除元素）。

  对于这种需求，最简单的解决方案便是将这样一组元素看成一个序列，用元素在序列里的位置和顺序，表示实际应用中的某种有意义的信息，或者表示数据之间的某种关系。

  这样的一组序列元素的组织形式，我们可以将其抽象为**线性表**。一个线性表是某类元素的一个集合，还记录着元素之间的一种顺序关系。线性表是最基本的数据结构之一，在实际程序中应用非常广泛，它还经常被用作更复杂的数据结构的实现基础。

  根据线性表的实际存储方式，分为两种实现模型：

  1. **顺序表**，将元素顺序地存放在一块连续的存储区里，元素间的顺序关系由它们的存储顺序自然表示。
  2. **链表**，将元素存放在通过链接构造起来的一系列存储块中。

---

## 3.1 顺序表的形式

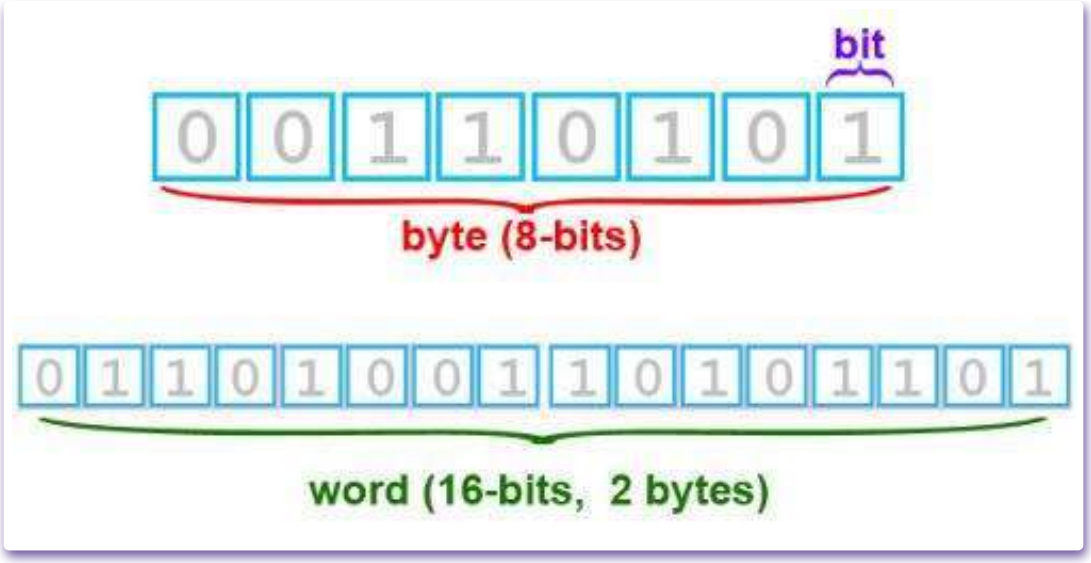
- 字节与字符

  一个字节（Byte）等于8位（bit，32位存储是4个字节，64位存储是8个字节。

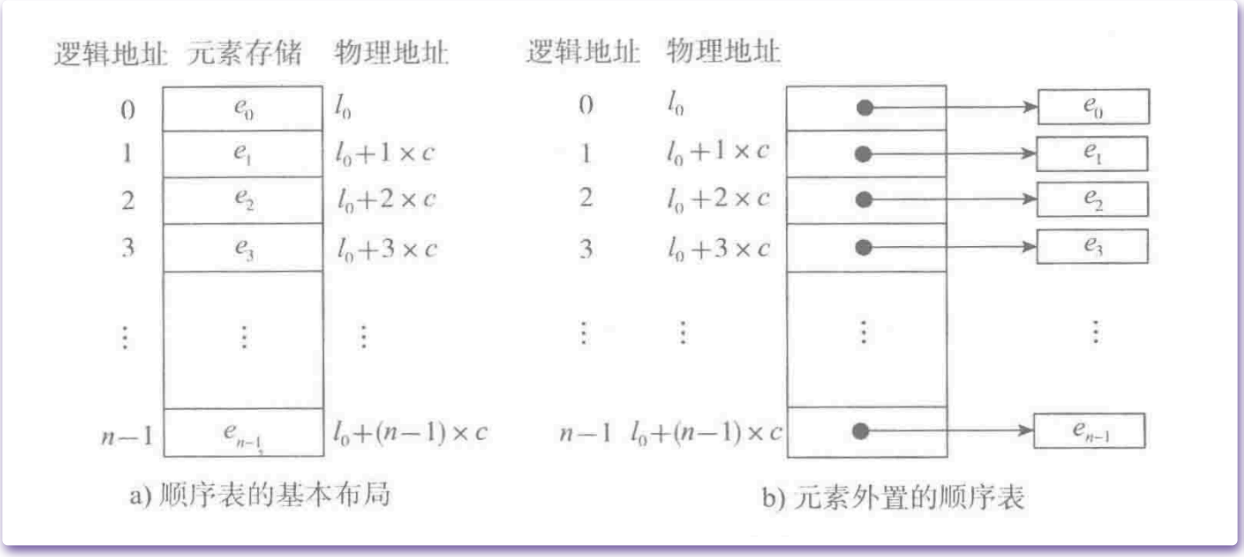
  字节(Byte)是计量单位，表示数据量多少，是计算机信息技术用于计量存储容量的一种计量单位，通常情况下一字节等于八位。

  字符(Character)计算机中使用的字母、数字、字和符号，比如'A'、'B'、'$'、'&'等。

```
一般在英文状态下一个字母或字符占用一个字节，一个汉字用两个字节表示。

ASCII 码中，一个英文字母（不分大小写）为一个字节，一个中文汉字为两个字节

UTF-8 编码中，一个英文字为一个字节，一个中文为三个字节。

Unicode 编码中，一个英文为一个字节，一个中文为两个字节。

符号：英文标点为一个字节，中文标点为两个字节。例如：英文句号 **.** 占1个字节的大小，中文句号 。占2个字节的大小。

UTF-16 编码中，一个英文字母字符或一个汉字字符存储都需要 2 个字节（Unicode 扩展区的一些汉字存储需要 4 个字节）。

UTF-32 编码中，世界上任何字符的存储都需要 4 个字节。
```




- 顺序表的基本形式




**图a**表示的是顺序表的基本形式，数据元素本身连续存储，每个元素所占的存储单元大小固定相同，元素的下标是其逻辑地址，而元素存储的物理地址（实际内存地址）可以通过存储区的起始地址Loc (e0)加上逻辑地址（第i个元素）与存储单元大小（c）的乘积计算而得，即：

**Loc(ei) = Loc(e0) + c*i**

故，访问指定元素时无需从头遍历，通过计算便可获得对应地址，其时间复杂度为O(1)。

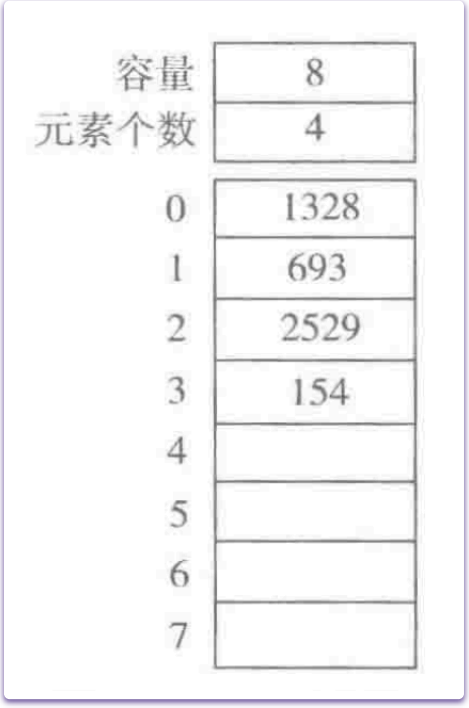
如果元素的大小不统一，则须采用图b的元素外置的形式，将实际数据元素另行存储，而顺序表中各单元位置保存对应元素的地址信息（即链接）。由于每个链接所需的存储量相同，通过上述公式，可以计算出元素链接的存储位置，而后顺着链接找到实际存储的数据元素。**注意，图b中的c不再是数据元素的大小，而是存储一个链接地址所需的存储量，这个量通常很小。**

图b这样的顺序表也被称为对实际数据的索引，这是最简单的索引结构。

---

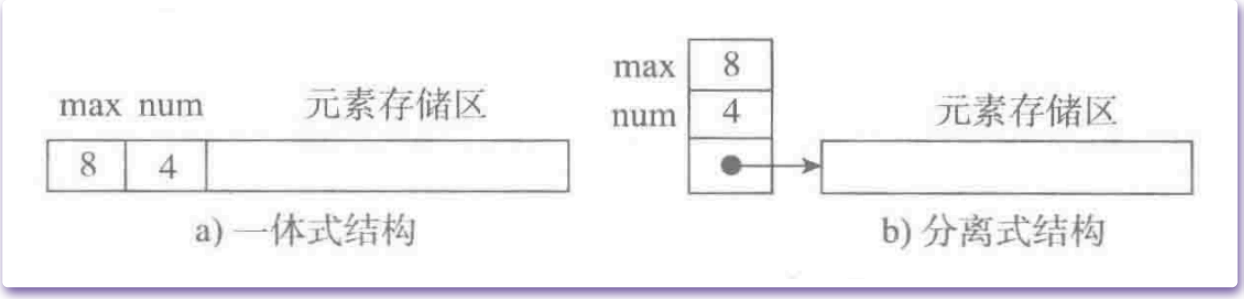
## 3.2 顺序表的结构与实现

- 顺序表的结构




一个顺序表的完整信息包括两部分，一部分是表中的元素集合，另一部分是为实现正确操作而需记录的信息，即有关表的整体情况的信息，这部分信息主要包括元素存储区的 **容量** 和当前表中已有的 **元素个数** 两项。（表头信息）

- 顺序表的两种基本实现方式




**图a**为 一体式结构 ，存储表信息的单元与元素存储区以连续的方式安排在一块存储区里，两部分数据的整体形成一个完整的顺序表对象。

一体式结构整体性强，易于管理。但是由于数据元素存储区域是表对象的一部分，顺序表创建后，元素存储区就固定了。

**图b**为 分离式结构（动态顺序表） ，表对象里只保存与整个表有关的信息（即容量和元素个数），实际数据元素存放在另一个独立的元素存储区里，通过链接与基本表对象关联。（分离式的表头除了要有容量和元素个数之外，还需要保存数据区的地址）

- 元素存储区替换

**一体式结构** 由于顺序表信息区与数据区连续存储在一起，所以**若想更换数据区，则只能整体搬迁**，即整个顺序表对象（指存储顺序表的结构信息的区域）改变了。

**分离式结构** 若想更换数据区，只需将表信息区中的数据区链接地址更新即可，而该顺序表对象不变。

- 元素存储区扩充

采用分离式结构的顺序表，若将数据区更换为存储空间更大的区域，则可以在不改变表对象的前提下对其数据存储区进行了扩充，所有使用这个表的地方都不必修改。只要程序的运行环境（计算机系统）还有空闲存储，这种表结构就不会因为满了而导致操作无法进行。人们把采用这种技术实现的顺序表称为动态顺序表，因为其容量可以在使用中动态变化。

**扩充的两种策略**

- 每次扩充增加固定数目的存储位置，如每次扩充增加10个元素位置，这种策略可称为线性增长。

  特点：节省空间，但是扩充操作频繁，操作次数多。

- 每次扩充容量加倍，如每次扩充增加一倍存储空间。

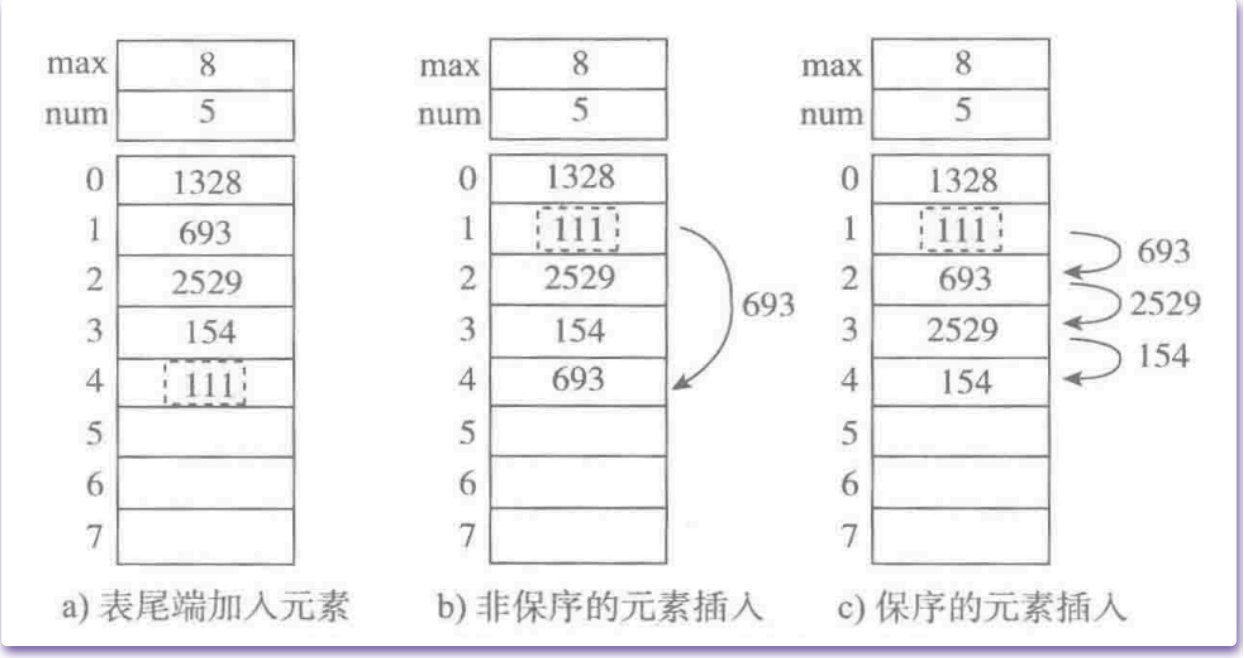
  特点：减少了扩充操作的执行次数，但可能会浪费空间资源。以空间换时间，推荐的方式。

---

## 3.3 顺序表的操作

- 增加元素

  如图所示，为顺序表增加新元素111的三种方式

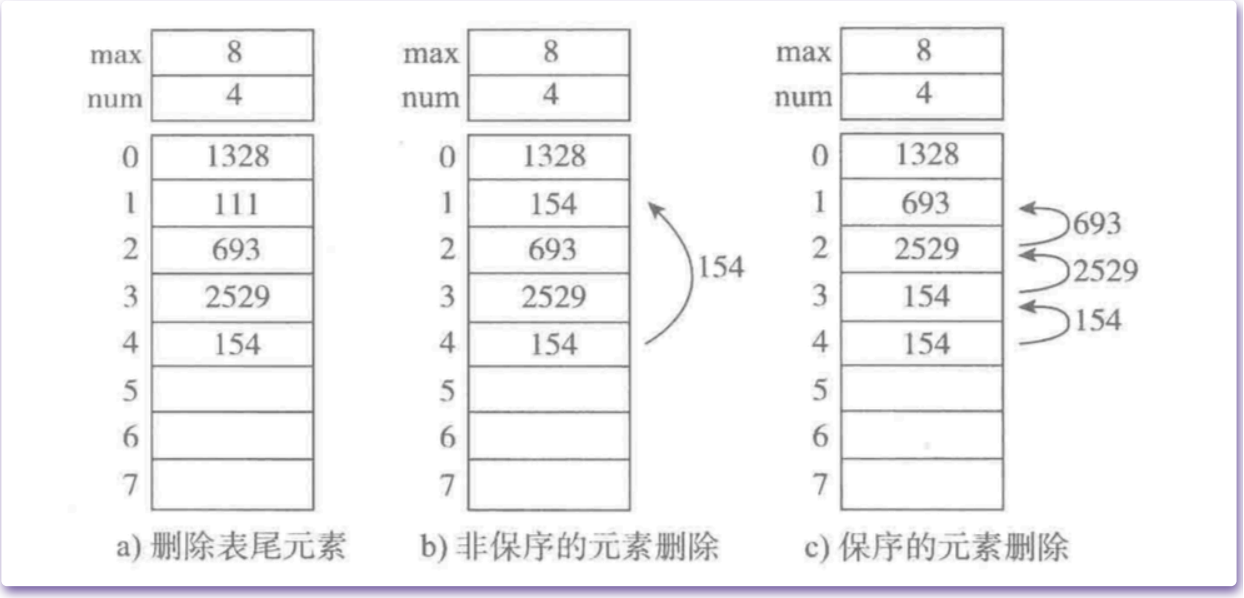



  a. 尾端加入元素，时间复杂度为O(1)

  b. 非保序的加入元素（不常见），时间复杂度为O(1)

  c. 保序的元素加入，时间复杂度为O(n)

- 删除元素




a. 删除表尾元素，时间复杂度为O(1)

b. 非保序的元素删除（不常见），时间复杂度为O(1)

c. 保序的元素删除，时间复杂度为O(n)

---

## 3.4 Python中的顺序表

- list与tuple

  Python中的list和tuple两种类型采用了顺序表的实现技术，具有前面讨论的顺序表的所有性质。

  tuple是不可变类型，即不变的顺序表，因此不支持改变其内部状态的任何操作，而其他方面，则与list的性质类似。

- list的基本实现技术

  Python标准类型list就是一种元素个数可变的线性表，可以加入和删除元素，并在各种操作中维持已有元素的顺序（即保序），而且还具有以下行为特征：

  - 基于下标（位置）的高效元素访问和更新，时间复杂度应该是O(1)；

    为满足该特征，应该采用顺序表技术，表中元素保存在一块连续的存储区中。

  - 允许任意加入元素，而且在不断加入元素的过程中，表对象的标识（函数id得到的值）不变。

    为满足该特征，就必须能更换元素存储区，并且为保证更换存储区时list对象的标识id不变，只能采用分离式实现技术。

  在Python的官方实现中，**list就是一种采用分离式技术实现的动态顺序表**。这就是为什么用list.append(x)（或list.insert(len(list), x)，即尾部插入）比在指定位置插入元素效率高的原因。

  在Python的官方实现中，list实现采用了如下的策略：在建立空表（或者很小的表）时，系统分配一块能容纳8个元素的存储区；在执行插入操作（insert或append）时，如果元素存储区满就换一块4倍大的存储区。但如果此时的表已经很大（目前的阀值为50000），则改

变策略，采用加一倍的方法。引入这种改变策略的方式，是为了避免出现过多空闲的存储位置。

# 4 三、链表（线性表）

- 为什么需要链表

  顺序表的构建需要预先知道数据大小来申请连续的存储空间，而在进行扩充时又需要进行数据的搬迁，所以使用起来并不是很灵活。

  链表结构可以充分利用计算机内存空间，实现灵活的内存动态管理。

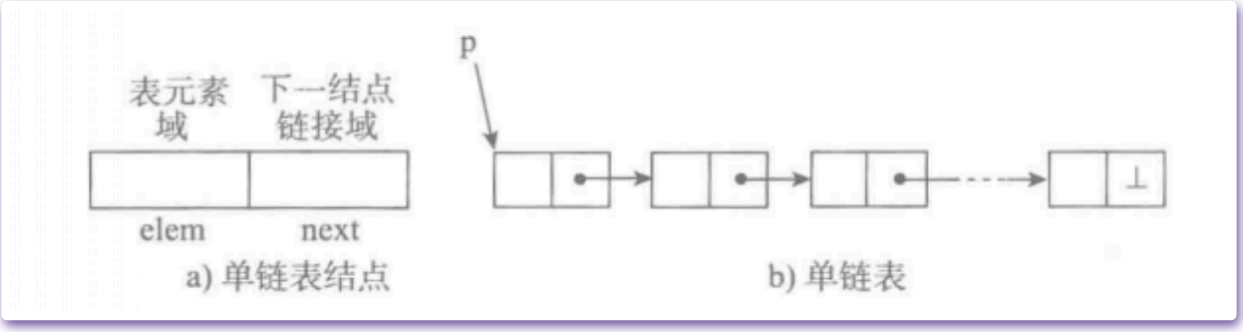
- 链表的定义

  链表（Linked list）是一种常见的基础数据结构，是一种线性表，但是不像顺序表一样连续存储数据，而是在每一个节点（数据存储单元）里存放下一个节点的位置信息（即地址）。

---

## 4.1 单向链表

- 单向链表也叫单链表，是链表中最简单的一种形式，它的每个节点包含两个域，一个信息域（元素域）和一个链接域。这个链接指向链表中的下一个节点，而最后一个节点的链接域则指向一个空值。

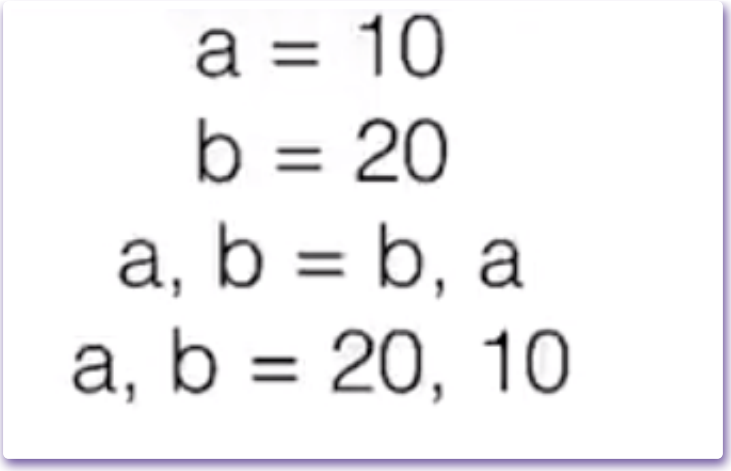



  - 表元素域**elem**用来存放具体的数据。
  - 链接域next用来存放下一个节点的位置（python中的标识）
  - **变量p**指向链表的头节点（首节点）的位置，从p出发能找到表中的任意节点。

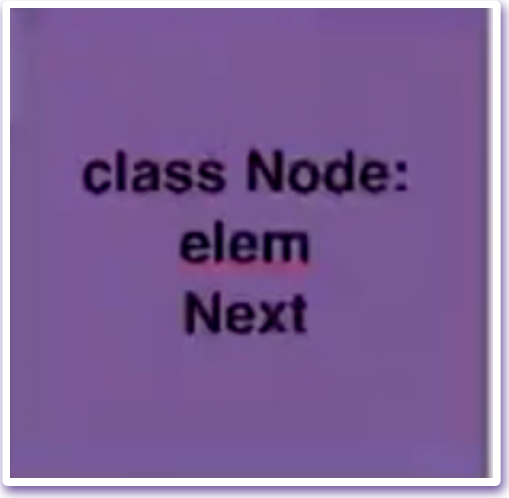
- **单链表节点实现**

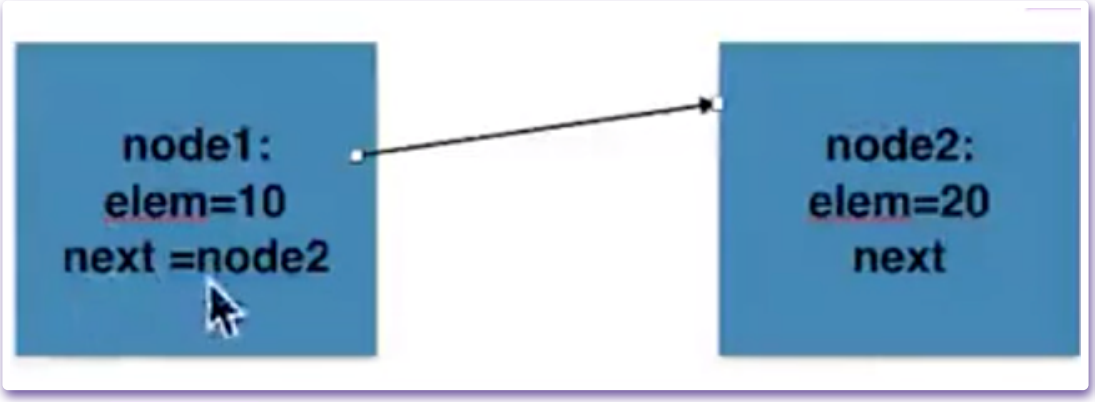
```python
class Node(object):
    """单链表的节点"""

    def __init__(self, elem):
        # _item存放数据元素
        self.elem = elem
        # _next 是下一个节点的标识
        self.next = None
```

- **python中变量标识的本质**




- 让a = 10 的时候，实际上指的是让a维护的那块区域 保存 存储数字10的那片区域的内存地址。
- a,b = b,a 的时候，实际上存储数字的区域不变，变的是a,b分别保存的内存地址




- 另**变量** ***p=Node()*** 实际上就是将node节点的elem(数据)和next（下一个节点）保存下来了（**将p指向链表的头结点的位置**）

- **单链表的操作**
  - is_empty() 链表是否为 空
  - **length()** 链表 长度
  - **travel()** 遍历 整个链表

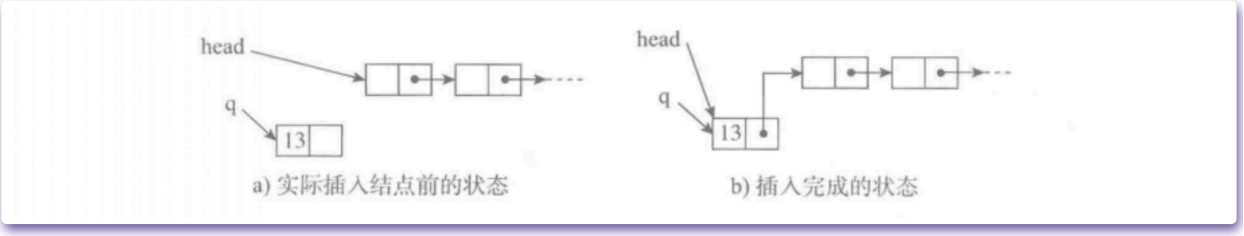
- add(item) 链表 头部添加 元素
- append(item) 链表 尾部添加 元素
- insert(pos, item) 指定位置添加 元素
- remove(item) 删除 节点
- search(item) 查找节点 是否存在

- 单链表的实现

```python
class SingleLinkList(object):
    """单链表"""

    """如果用下面的这个构造函数，意思是一上来创建的是一个空链表，表头指向的是None"""
    # def __init__(self):
    #     self._head = None
    """如果用下面这个构造函数，意思就是一上来得先创建一个节点，然后让表头指向这个节点，即
    创建的链表不是空链表"""
    # def __init__(self, node):
    #     self._head = node

    """如果用下面这个构造函数，默认上来创建的是一个空链表"""

    def __init__(self, node=None):
        self.__head = node

    def is_empty(self):
        """链表是否为空"""
        return self.__head == None

    def length(self):
        """链表长度"""
        # cur 游标，用来移动遍历节点
        cur = self.__head
        # count 记录数量
        count = 0
        while cur != None:  # 如果是空链表，则一上来进不了循环，count直接=0，满足特殊条件
            count += 1
            cur = cur.next
        return count

    def travel(self):
        """遍历整个链表"""
        cur = self.__head
        while cur != None:
            print(cur.elem, end=' ')
            cur = cur.next
        # 以上也是可以处理当链表为空链表时的特殊情况
```

```
        print('')  # 起换行作用
```

- **单向链表头部添加元素**




```
    def add(self, item):
        """链表头部添加元素，头插法"""
        # 先创建一个保存item值的节点
        node = Node(item)
        # 先将新节点的链接域next指向头节点，即_head指向的位置
        node.next = self.__head
        # 再将链表的头_head指向新节点
        self.__head = node
```

- **单向链表尾部添加元素**

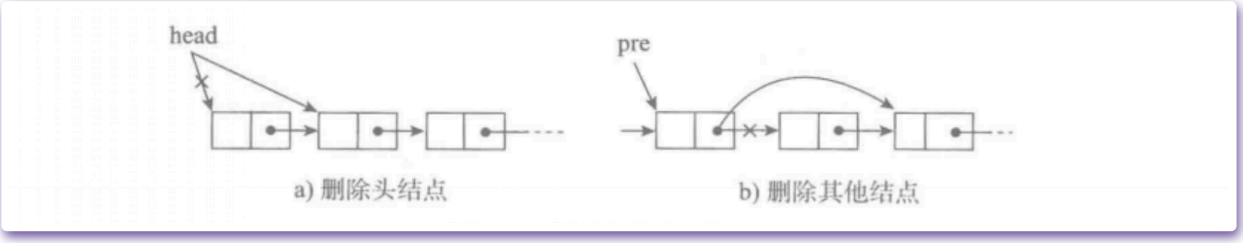
```
    def append(self, item):
        """链表尾部添加元素,尾插法"""
        node = Node(item)
        # 先判断链表是否为空，若是空链表，则将_head指向新节点
        if self.is_empty():
            self.__head = node
        # 若不为空，则找到尾部，将尾节点的next指向新节点
        else:
            cur = self.__head
            while cur.next != None:
                cur = cur.next
            cur.next = node
```

- **单向链表指定位置添加元素**

```
    def insert(self, pos, item):
        """指定位置添加元素
        :param   pos   从0开始
        """
        # 若指定位置pos为第一个元素之前，则执行头部插入
        if pos <= 0:
            self.add(item)  # 如果pos<0，这里认为直接是头部插入
```

```python
        # 若指定位置超过链表尾部，则执行尾部插入
        elif pos > (self.length() - 1):
            self.append(item)
        # 找到指定位置
        else:
            # pre用来指向指定位置pos的前一个位置pos-1，初始从头节点开始移动到指定位置
            pre = self.__head
            count = 0
            while count < pos - 1:
                count += 1
                pre = pre.next
            # 当循环退出后，pre指向pos-1位置
            node = Node(item)
            # 先将新节点node的next指向插入位置的节点
            node.next = pre.next
            # 将插入位置的前一个节点的next指向新节点
            pre.next = node
```

- **单向链表删除节点**




```python
    def remove(self, item):
        """删除节点，为了方便好理解，用两个游标"""
        cur = self.__head
        pre = None  # pre跟着cur的前一个
        while cur != None:
            # 找到指定元素
            if cur.elem == item:
                # 先判断此节点是不是头节点
                # 头节点
                if cur == self.__head:
                    self.__head = cur.next
                else:
                    # 将删除位置前一个节点的next指向删除位置的后一个节点
                    pre.next = cur.next
                break
            else:
                # 继续按链表后移节点
                pre = cur
                cur = cur.next
```

- **单向链表查找节点是否存在**

```python
    def search(self, item):
        """查找节点是否存在"""
        cur = self.__head
        while cur != None:
            if cur.elem == item:
                return True
            else:
                cur = cur.next
        return False
        # 上述代码若是空链表，直接返回False，也能满足条件
```

- 单向链表测试

```python
def main():
    ll = SingleLinkList()
    print(ll.is_empty())
    print(ll.length())
    ll.append(1)
    print(ll.is_empty())
    print(ll.length())
    ll.append(2)
    ll.add(8)
    ll.append(3)
    ll.append(4)
    ll.append(5)
    ll.append(6)
    # 8 1 2 3 4 5 6
    ll.insert(-1, 9)
    ll.travel()  # 9 8 1 2 3 4 5 6
    ll.insert(3, 100)
    ll.travel()  # 9 8 1 100 2 3 4 5 6
    ll.insert(10, 200)
    ll.travel()  # 9 8 1 100 2 3 4 5 6 200
    ll.remove(100)
    ll.travel()  # 9 8 1 2 3 4 5 6 200
    ll.remove(9)
    ll.travel()  # 8 1 2 3 4 5 6 200
    ll.remove(200)
    ll.travel()  # 8 1 2 3 4 5 6


if __name__ == "__main__":
    main()

```

- 单向链表代码集合

```python
# -*- coding: utf-8 -*-

"""
**************************************************
@author:   Ying
```

```python
@software: PyCharm
@file: 07、single_link_list.py
@time: 2021/2/23 18:03
**************************************************
"""


class Node(object):
    """单链表的节点"""

    def __init__(self, elem):
        # _item存放数据元素
        self.elem = elem
        # _next 是下一个节点的标识
        self.next = None


class SingleLinkList(object):
    """单链表"""

    """如果用下面的这个构造函数，意思是一上来创建的是一个空链表，表头指向的是None"""
    # def __init__(self):
    #     self._head = None
    """如果用下面这个构造函数，意思就是一上来得先创建一个节点，然后让表头指向这个节点，即
    创建的链表不是空链表"""
    # def __init__(self, node):
    #     self._head = node

    """如果用下面这个构造函数，默认上来创建的是一个空链表"""

    def __init__(self, node=None):
        self.__head = node

    def is_empty(self):
        """链表是否为空"""
        return self.__head == None

    def length(self):
        """链表长度"""
        # cur 游标，用来移动遍历节点，初始时指向头节点
        cur = self.__head
        # count 记录数量
        count = 0
        while cur != None:  # 如果是空链表，则一上来进不了循环，count直接=0，满足特殊条件
            count += 1
            # 将cur后移一个节点
            cur = cur.next
        return count

```

```python
    def travel(self):
        """遍历整个链表"""
        cur = self.__head
        while cur != None:
            print(cur.elem, end=' ')
            cur = cur.next
        # 以上也是可以处理当链表为空链表时的特殊情况
        print('')  # 起换行作用

    def add(self, item):
        """链表头部添加元素，头插法"""
        # 先创建一个保存item值的节点
        node = Node(item)
        # 先将新节点的链接域next指向头节点，即_head指向的位置
        node.next = self.__head
        # 再将链表的头_head指向新节点
        self.__head = node

    def append(self, item):
        """链表尾部添加元素,尾插法"""
        node = Node(item)
        # 先判断链表是否为空，若是空链表，则将_head指向新节点
        if self.is_empty():
            self.__head = node
        # 若不为空，则找到尾部，将尾节点的next指向新节点
        else:
            cur = self.__head
            while cur.next != None:
                cur = cur.next
            cur.next = node

    def insert(self, pos, item):
        """指定位置添加元素
        :param   pos  从0开始
        """
        # 若指定位置pos为第一个元素之前，则执行头部插入
        if pos <= 0:
            self.add(item)  # 如果pos<0，这里认为直接是头部插入
        # 若指定位置超过链表尾部，则执行尾部插入
        elif pos > (self.length() - 1):
            self.append(item)
        # 找到指定位置
        else:
            # pre用来指向指定位置pos的前一个位置pos-1，初始从头节点开始移动到指定位置
            pre = self.__head
            count = 0
            while count < pos - 1:
                count += 1
                pre = pre.next
            # 当循环退出后，pre指向pos-1位置
            node = Node(item)
```

```python
            # 先将新节点node的next指向插入位置的节点
            node.next = pre.next
            # 将插入位置的前一个节点的next指向新节点
            pre.next = node

    def remove(self, item):
        """删除节点，为了方便好理解，用两个游标"""
        cur = self.__head
        pre = None  # pre跟着cur的前一个
        while cur != None:
            # 找到指定元素
            if cur.elem == item:
                # 先判断此节点是不是头节点
                # 头节点
                if cur == self.__head:
                    self.__head = cur.next
                else:
                    # 将删除位置前一个节点的next指向删除位置的后一个节点
                    pre.next = cur.next
                break
            else:
                # 继续按链表后移节点
                pre = cur
                cur = cur.next

    def search(self, item):
        """查找节点是否存在"""
        cur = self.__head
        while cur != None:
            if cur.elem == item:
                return True
            else:
                cur = cur.next
        return False
        # 上述代码若是空链表，直接返回False，也能满足条件


def main():
    ll = SingleLinkList()
    print(ll.is_empty())
    print(ll.length())
    ll.append(1)
    print(ll.is_empty())
    print(ll.length())
    ll.append(2)
    ll.add(8)
    ll.append(3)
    ll.append(4)
    ll.append(5)
    ll.append(6)
    # 8 1 2 3 4 5 6
    ll.insert(-1, 9)
```

```
    ll.travel()  # 9 8 1 2 3 4 5 6
    ll.insert(3, 100)
    ll.travel()  # 9 8 1 100 2 3 4 5 6
    ll.insert(10, 200)
    ll.travel()  # 9 8 1 100 2 3 4 5 6 200
    ll.remove(100)
    ll.travel()  # 9 8 1 2 3 4 5 6 200
    ll.remove(9)
    ll.travel()  # 8 1 2 3 4 5 6 200
    ll.remove(200)
    ll.travel()  # 8 1 2 3 4 5 6


if __name__ == "__main__":
    main()
```

## 4.2 链表与顺序表的对比

- 链表失去了顺序表随机读取的优点，同时链表由于增加了结点的指针域，空间开销比较大，但对存储空间的使用要相对灵活。

  链表与顺序表的各种操作复杂度如下所示：

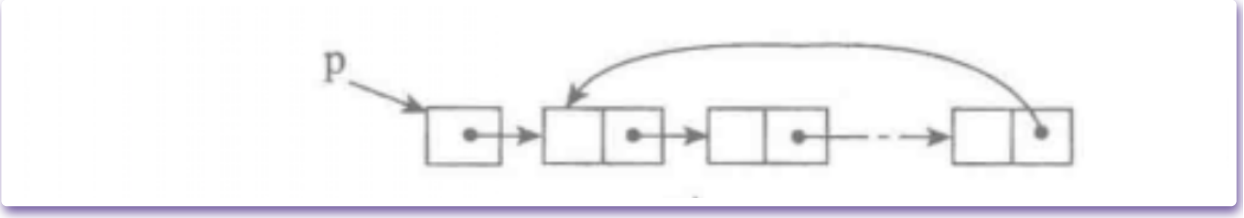
| 操作 | 链表 | 顺序表 |
|---|---|---|
| 访问元素 | O(n) | O(1) |
| 在头部插入/删除 | O(1) | O(n) |
| 在尾部插入/删除 | O(n) | O(1) |
| 在中间插入/删除 | O(n) | O(n) |

注意虽然表面看起来复杂度都是 O(n)，但是**链表和顺序表在插入和删除时进行的是完全不同的操作**。链表的主要耗时操作是遍历查找，删除和插入操作本身的复杂度是O(1)。顺序表查找很快，主要耗时的操作是拷贝覆盖。因为除了目标元素在尾部的特殊情况，顺序表进行插入和删除时需要对操作点之后的所有元素进行前后移位操作，只能通过拷贝和覆盖的方法进行。

## 4.3 单向循环链表

- 单向循环链表

单链表的一个变形是单向循环链表，链表中最后一个节点的next域不再为None，而是指向链表的头节点。




- 单向循环链表操作
  - is_empty() 链表是否为 空
  - length() 链表 长度
  - travel() 遍历 整个链表
  - add(item) 链表 头部添加 元素
  - append(item) 链表 尾部添加 元素
  - insert(pos, item) 指定位置添加 元素
  - remove(item) 删除 节点
  - search(item) 查找节点 是否存在
- 单向循环链表的实现

```python
# -*- coding: utf-8 -*-

"""
**************************************************
@author:   Ying
@software: PyCharm
@file: 09、single_cycle_link_list.py

@time: 2021/2/26 11:07
**************************************************
"""


class Node(object):
    """单向循环链表的节点"""

    def __init__(self, elem):
        # _item存放数据元素
        self.elem = elem
        # _next 是下一个节点的标识
        self.next = None


class SingleCycleLinkList(object):
    """单向循环链表"""

    def __init__(self, node=None):
```

```python
        self.__head = node
        # 或者if node:
        if node is not None:
            # 如果传进来的节点不是空的，那么因为是单向循环链表，还得将这个、
            # 节点的next指向头节点，而此时的头节点就是node
            node.next = node

    def is_empty(self):
        """链表是否为空"""
        return self.__head is None

    def length(self):
        """链表长度"""
        if self.is_empty():
            return 0
        # cur 游标，用来移动遍历节点，初始时指向头节点
        cur = self.__head
        # count 记录数量
        count = 1
        while cur.next != self.__head:  # 如果是空链表，则一上来进不了循环，count直接=0，满足特殊条件
            count += 1
            # 将cur后移一个节点
            cur = cur.next
        return count

    def travel(self):
        """遍历整个单向循环链表"""
        if self.is_empty():
            return
        cur = self.__head
        while cur.next != self.__head:
            print(cur.elem, end=' ')
            cur = cur.next
        # 退出循环的时候，cur指向尾结点，但是尾结点的元素未打印，所以需要主动打印一下
        # 这个打印，也将只有一个元素的单向循环链表打印出来的情况包含在内
        print(cur.elem)
        print('')  # 起换行作用

    def add(self, item):
        """单向循环链表头部添加元素，头插法"""
        # 先创建一个保存item值的节点
        node = Node(item)
        if self.is_empty():
            self.__head = node
            node.next = node
        else:  # 或者下面的不用放在else里面，在上一行 node.next = node 后直接return也行
            cur = self.__head
            while cur.next != self.__head:
                cur = cur.next
```

```
            # 退出循环，cur指向的是尾结点
            node.next = self.__head
            self.__head = node
            # cur.next = node  下一句或者这个也行
            cur.next = self.__head

    def append(self, item):
        """单向循环链表尾部添加元素,尾插法"""
        node = Node(item)
        # 先判断链表是否为空，若是空链表，则将_head指向新节点
        if self.is_empty():
            self.__head = node
            node.next = node
        # 若不为空，则找到尾部，将尾节点的next指向新节点
        else:
            cur = self.__head
            while cur.next != self.__head:
                cur = cur.next

            # cur.next = node
            # node.next = self.__head
            # 或者
            node.next = cur.next
            cur.next = node

    def insert(self, pos, item):
        """单向循环链表指定位置添加元素
        :param   pos  从0开始
        """
        # 若指定位置pos为第一个元素之前，则执行头部插入
        if pos <= 0:
            self.add(item)  # 如果pos<0，这里认为直接是头部插入
        # 若指定位置超过链表尾部，则执行尾部插入
        elif pos > (self.length() - 1):
            self.append(item)
        # 找到指定位置
        else:
            # pre用来指向指定位置pos的前一个位置pos-1，初始从头节点开始移动到指定位置
            pre = self.__head
            count = 0
            while count < pos - 1:
                count += 1
                pre = pre.next
            # 当循环退出后，pre指向pos-1位置
            node = Node(item)
            # 先将新节点node的next指向插入位置的节点
            node.next = pre.next
            # 将插入位置的前一个节点的next指向新节点
            pre.next = node

    def remove(self, item):
```

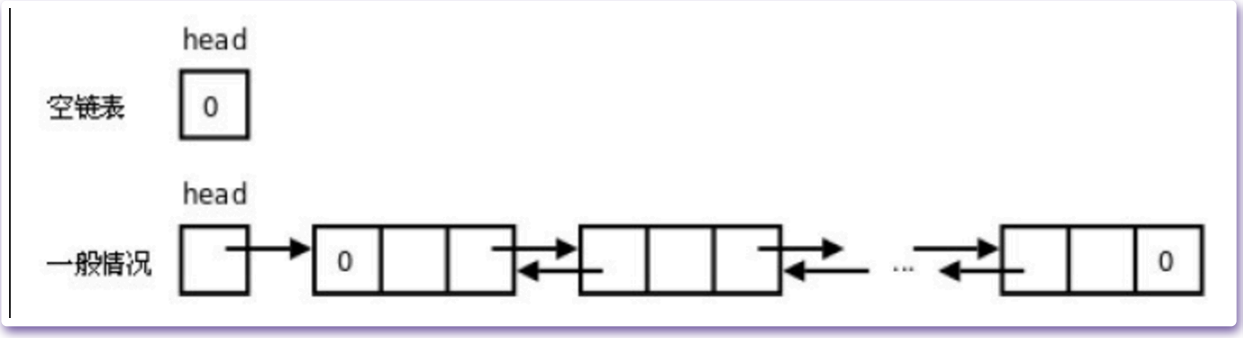
```python
        """单向循环链表删除节点，为了方便好理解，用两个游标"""
        if self.is_empty():
            return

        cur = self.__head
        pre = None  # pre跟着cur的前一个

        while cur.next != self.__head:
            # 找到指定元素
            if cur.elem == item:
                # 先判断此节点是不是头节点
                # 头节点
                if cur == self.__head:
                    # 头节点的情况
                    # 找尾节点
                    rear = self.__head
                    while rear.next != self.__head:
                        rear = rear.next
                    self.__head = cur.next
                    rear.next = self.__head
                else:
                    # 中间节点
                    # 将删除位置前一个节点的next指向删除位置的后一个节点
                    pre.next = cur.next
                return
            else:
                # 继续按链表后移节点
                pre = cur
                cur = cur.next
        # 退出循环，cur指向尾结点
        if cur.elem == item:
            if cur == self.__head:
                # 链表当中只有一个节点，即首节点就是尾结点
                self.__head = None
            else:
                pre.next = cur.next
                # 或者
                # pre.next = self.__head

    def search(self, item):
        """单向循环链表查找节点是否存在"""
        if self.is_empty():
            return False
        cur = self.__head
        while cur.next != self.__head:
            if cur.elem == item:
                return True
            else:
                cur = cur.next
        # 退出循环，cur指向为节点
        if cur.item == item:
            return True
```

```
        return False
        # 上述代码若是空链表，直接返回False，也能满足条件


def main():
    ll = SingleCycleLinkList()
    print(ll.is_empty())
    print(ll.length())
    ll.append(1)
    print(ll.is_empty())
    print(ll.length())
    ll.append(2)
    ll.add(8)
    ll.append(3)
    ll.append(4)
    ll.append(5)
    ll.append(6)
    # 8 1 2 3 4 5 6
    ll.insert(-1, 9)
    ll.travel()  # 9 8 1 2 3 4 5 6
    ll.insert(3, 100)
    ll.travel()  # 9 8 1 100 2 3 4 5 6
    ll.insert(10, 200)
    ll.travel()  # 9 8 1 100 2 3 4 5 6 200
    ll.remove(100)
    ll.travel()  # 9 8 1 2 3 4 5 6 200
    ll.remove(9)
    ll.travel()  # 8 1 2 3 4 5 6 200
    ll.remove(200)
    ll.travel()  # 8 1 2 3 4 5 6
    print(ll.length())


if __name__ == "__main__":
    main()

```

# 4.4 双向链表

- **双向链表**

  一种更复杂的链表是“双向链表”或“双面链表”。每个节点有两个链接：一个指向前一个节点，当此节点为第一个节点时，指向空值；而另一个指向下一个节点，当此节点为最后一个节点时，指向空值。




- 双向链表的操作
  - is_empty() 链表是否为 空
  - length() 链表 长度
  - travel() 遍历 整个链表
  - add(item) 链表 头部添加 元素
  - append(item) 链表 尾部添加 元素
  - insert(pos, item) 指定位置添加 元素
  - remove(item) 删除 节点
  - search(item) 查找节点 是否存在
- 双向链表的节点实现

```
class Node(object):
    """双向链表节点"""

    def __init__(self, item):
        self.elem = item
        self.prev = None  # 头节点的前驱指向为空
        self.next = None  # 尾结点的后继指向为空
```

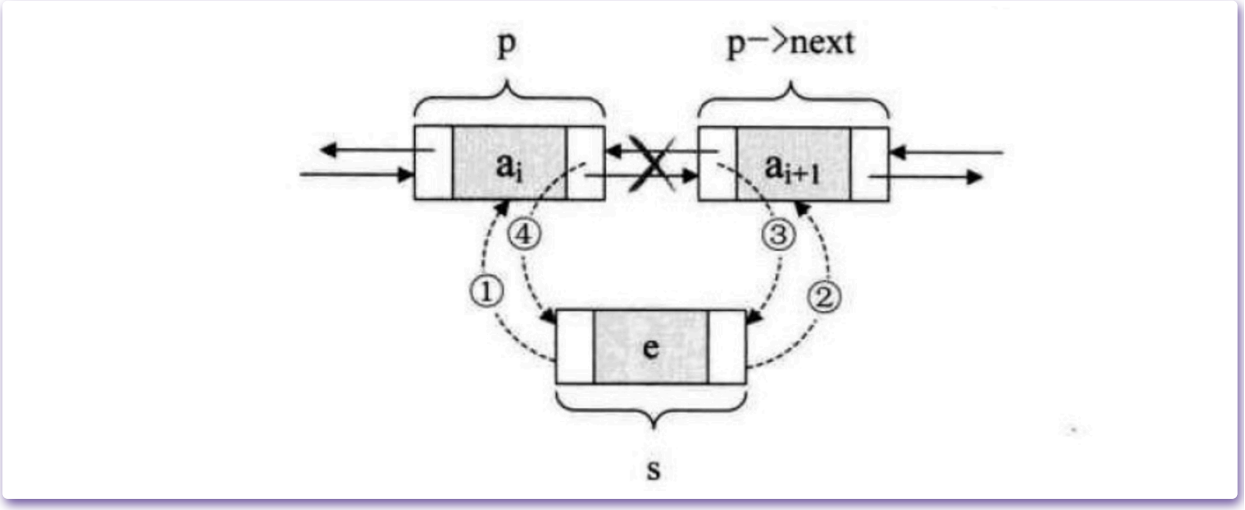
- 双向链表的实现

```
class DoubleLinkList(object):
    """双向链表"""

    def __init__(self, node=None):
        self.__head = node

    def is_empty(self):
        """判断双链表是否为空"""
        return self.__head is None

    def length(self):
        """双链表长度"""
        # cur 游标，用来移动遍历节点，初始时指向头节点
        cur = self.__head
        # count 记录数量
        count = 0
```

```python
        while cur is not None:  # 如果是空链表，则一上来进不了循环，count直接=0，满足特殊条件
            count += 1
            # 将cur后移一个节点
            cur = cur.next
        return count

    def travel(self):
        """遍历整个双链表"""
        cur = self.__head
        while cur is not None:
            print(cur.elem, end=' ')
            cur = cur.next
        # 以上也是可以处理当链表为空链表时的特殊情况
        print('')  # 起换行作用

    def add(self, item):
        """双链表头部添加元素，头插法"""
        # 先创建一个保存item值的节点
        node = Node(item)
        if self.is_empty():
            # 如果是空链表，将self.__head 直接指向node
            self.__head = node
        else:
            # 先将新节点的链接域next指向头节点，即_head指向的位置
            node.next = self.__head
            # 再将链表的头_head指向新节点
            self.__head = node
            node.next.prev = node
        """注意，此处else下面也可以这样写，写法可能不唯一

         # 将node的next指向_head的头节点
            node.next = self._head
            # 将_head的头节点的prev指向node
            self._head.prev = node
            # 将_head 指向node
            self._head = node
        """

    def append(self, item):
        """双链表尾部添加元素,尾插法"""
        node = Node(item)
        # 先判断链表是否为空，若是空链表，则将_head指向新节点
        if self.is_empty():
            self.__head = node
        # 若不为空，则找到尾部，将尾节点的next指向新节点
        else:
            cur = self.__head
            # while 循环是将 cur 移动到链表尾部
            while cur.next is not None:
                cur = cur.next
            # 将尾节点cur的next指向node
```

```
            cur.next = node
            # 将node的prev指向cur
            node.prev = cur
    def search(self, item):
        """查找元素是否存在"""
        cur = self.__head
        while cur is not None:
            if cur.elem == item:
                return True
            else:
                cur = cur.next
        return False
        # 上述代码若是空链表，直接返回False，也能满足条件
```

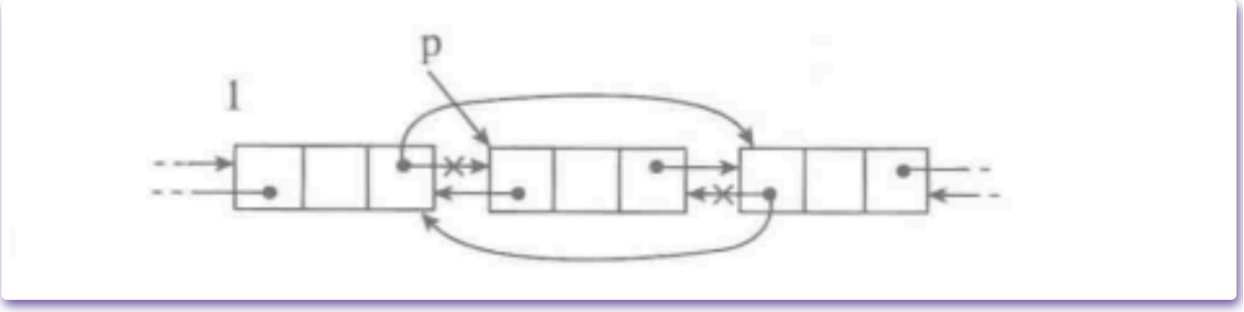
- 双向链表指定位置插入节点




```
    def insert(self, pos, item):
        """
        指定位置添加元素
        :param   pos   从0开始
        """
        # 若指定位置pos为第一个元素之前，则执行头部插入
        if pos <= 0:
            self.add(item)  # 如果pos<0，这里认为直接是头部插入
        # 若指定位置超过链表尾部，则执行尾部插入
        elif pos > (self.length() - 1):
            self.append(item)
        # 找到指定位置
        else:
            cur = self.__head
            count = 0
            while count < pos:
                count += 1
                cur = cur.next
            # 当循环退出后，cur指向pos位置
            node = Node(item)

```

```
            node.next = cur
            node.prev = cur.prev
            cur.prev.next = node
            cur.prev = node
            """上面四句也可以改成
            node.next = cur
            node.prev = cur.prev
            cur.prev = node
            node.prev.next = node
            """
```

- 双向链表删除元素




```
def remove(self, item):
    """删除节点"""
    cur = self.__head
    while cur is not None:
        # 找到指定元素
        if cur.elem == item:
            # 先判断此节点是不是头节点
            # 头节点
            if cur == self.__head:
                self.__head = cur.next
                # 或者直接 if cur.next
                if cur.next is not None:
                    # 判断双链表是否只有一个结点，如果只有一个节点，那么cur.next 是None，
                    # 这时候cur.next 是没有 prev的
                    cur.next.prev = None
            else:
                cur.prev.next = cur.next
                # 或者直接 if cur.next
                if cur.next is not None:
                    cur.next.prev = cur.prev
            break
        else:
            # 继续按链表后移节点
            cur = cur.next
```

- 双向链表测试

```
def run():
    ll = DoubleLinkList()
```

```
    print(ll.is_empty())
    print(ll.length())
    ll.append(1)
    print(ll.is_empty())
    print(ll.length())
    ll.append(2)
    ll.add(8)
    ll.append(3)
    ll.append(4)
    ll.append(5)
    ll.append(6)
    # 8 1 2 3 4 5 6
    ll.insert(-1, 9)
    ll.travel()  # 9 8 1 2 3 4 5 6
    ll.insert(3, 100)
    ll.travel()  # 9 8 1 100 2 3 4 5 6
    ll.insert(10, 200)
    ll.travel()  # 9 8 1 100 2 3 4 5 6 200
    ll.remove(100)
    ll.travel()  # 9 8 1 2 3 4 5 6 200
    ll.remove(9)
    ll.travel()  # 8 1 2 3 4 5 6 200
    ll.remove(200)
    ll.travel()  # 8 1 2 3 4 5 6


if __name__ == "__main__":
    run()
```

- 双向链表代码集合

```
# -*- coding: utf-8 -*-

"""
**************************************************
@author:   Ying
@software: PyCharm
@file: 08、double_link_list.py
@time: 2021/2/25 09:30
**************************************************
"""


class Node(object):
    """双向链表节点"""

    def __init__(self, item):
        self.elem = item
        self.prev = None  # 头节点的前驱指向为空
        self.next = None  # 尾结点的后继指向为空


```

```python
class DoubleLinkList(object):
    """双向链表"""

    def __init__(self, node=None):
        self.__head = node

    def is_empty(self):
        """判断双链表是否为空"""
        return self.__head is None

    def length(self):
        """双链表长度"""
        # cur 游标，用来移动遍历节点，初始时指向头节点
        cur = self.__head
        # count 记录数量
        count = 0
        while cur is not None:  # 如果是空链表，则一上来进不了循环，count直接=0，满足特殊条件
            count += 1
            # 将cur后移一个节点
            cur = cur.next
        return count

    def travel(self):
        """遍历整个双链表"""
        cur = self.__head
        while cur is not None:
            print(cur.elem, end=' ')
            cur = cur.next
        # 以上也是可以处理当链表为空链表时的特殊情况
        print('')  # 起换行作用

    def add(self, item):
        """双链表头部添加元素，头插法"""
        # 先创建一个保存item值的节点
        node = Node(item)
        if self.is_empty():
            # 如果是空链表，将self.__head 直接指向node
            self.__head = node
        else:
            # 先将新节点的链接域next指向头节点，即_head指向的位置
            node.next = self.__head
            # 再将链表的头_head指向新节点
            self.__head = node
            node.next.prev = node
        """注意，此处else下面也可以这样写，写法可能不唯一

         # 将node的next指向_head的头节点
            node.next = self._head
            # 将_head的头节点的prev指向node
            self._head.prev = node
            # 将_head 指向node
```

```
            self._head = node
        """

    def append(self, item):
        """双链表尾部添加元素,尾插法"""
        node = Node(item)
        # 先判断链表是否为空，若是空链表，则将_head指向新节点
        if self.is_empty():
            self.__head = node
        # 若不为空，则找到尾部，将尾节点的next指向新节点
        else:
            cur = self.__head
            # while 循环是将 cur 移动到链表尾部
            while cur.next is not None:
                cur = cur.next
            # 将尾节点cur的next指向node
            cur.next = node
            # 将node的prev指向cur
            node.prev = cur

    def insert(self, pos, item):
        """
        指定位置添加元素
        :param   pos   从0开始
        """
        # 若指定位置pos为第一个元素之前，则执行头部插入
        if pos <= 0:
            self.add(item)  # 如果pos<0，这里认为直接是头部插入
        # 若指定位置超过链表尾部，则执行尾部插入
        elif pos > (self.length() - 1):
            self.append(item)
        # 找到指定位置
        else:
            cur = self.__head
            count = 0
            while count < pos:
                count += 1
                cur = cur.next
            # 当循环退出后，cur指向pos位置
            node = Node(item)

            node.next = cur
            node.prev = cur.prev
            cur.prev.next = node
            cur.prev = node
            """上面四句也可以改成
            node.next = cur
            node.prev = cur.prev
            cur.prev = node
            node.prev.next = node
            """

```

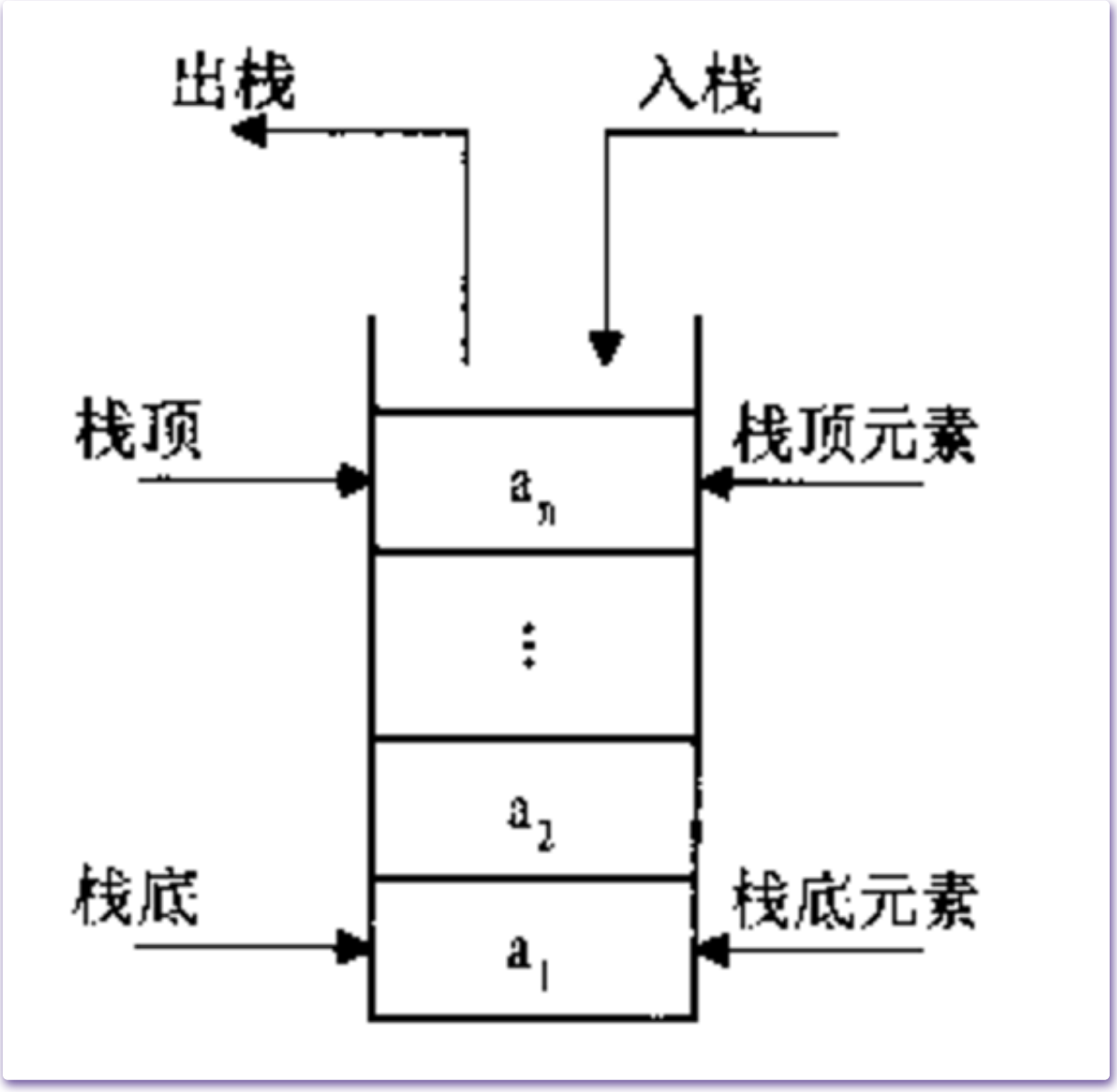
```python
    def remove(self, item):
        """删除节点"""
        cur = self.__head
        while cur is not None:
            # 找到指定元素
            if cur.elem == item:
                # 先判断此节点是不是头节点
                # 头节点
                if cur == self.__head:
                    self.__head = cur.next
                    # 或者直接 if cur.next
                    if cur.next is not None:
                        # 判断双链表是否只有一个结点，如果只有一个节点，那么cur.next 是None，
                        # 这时候cur.next 是没有 prev的
                        cur.next.prev = None
                else:
                    cur.prev.next = cur.next
                    # 或者直接 if cur.next
                    if cur.next is not None:
                        cur.next.prev = cur.prev
                break
            else:
                # 继续按链表后移节点
                cur = cur.next

    def search(self, item):
        """查找元素是否存在"""
        cur = self.__head
        while cur is not None:
            if cur.elem == item:
                return True
            else:
                cur = cur.next
        return False
        # 上述代码若是空链表，直接返回False，也能满足条件


def run():
    ll = DoubleLinkList()
    print(ll.is_empty())
    print(ll.length())
    ll.append(1)
    print(ll.is_empty())
    print(ll.length())
    ll.append(2)
    ll.add(8)
    ll.append(3)
    ll.append(4)
    ll.append(5)
    ll.append(6)
    # 8 1 2 3 4 5 6
```

```python
    ll.insert(-1, 9)
    ll.travel()  # 9 8 1 2 3 4 5 6
    ll.insert(3, 100)
    ll.travel()  # 9 8 1 100 2 3 4 5 6
    ll.insert(10, 200)
    ll.travel()  # 9 8 1 100 2 3 4 5 6 200
    ll.remove(100)
    ll.travel()  # 9 8 1 2 3 4 5 6 200
    ll.remove(9)
    ll.travel()  # 8 1 2 3 4 5 6 200
    ll.remove(200)
    ll.travel()  # 8 1 2 3 4 5 6


if __name__ == "__main__":
    run()

```

# 5 四、栈

- 栈

  栈（stack），有些地方称为堆栈，是一种容器，可存入数据元素、访问元素、删除元素，它的特点在于只能允许在容器的一端（称为栈顶端指标，英语：top）进行加入数据（英语：push）和输出数据（英语：pop）的运算。没有了位置概念，保证任何时候可以访问、删除的元素都是此前最后存入的那个元素，确定了一种默认的访问顺序。
- 由于栈数据结构只允许在一端进行操作，因而按照后进先出（LIFO, Last In First Out）的原理运作。




## 5.1 栈结构实现

- **栈**可以用**顺序表**实现，也可以用**链表**实现。
- **栈的操作**

    - Stack() 创建一个新的空栈
    - push(item) 添加一个新的元素item到栈顶
    - pop() 弹出栈顶元素
    - peek() 返回栈顶元素
    - is_empty() 判断栈是否为空
    - size() 返回栈的元素个数
- 栈结构代码

```python
# -*- coding: utf-8 -*-

"""
**************************************************
@author:   Ying
@software: PyCharm
@file: 10、栈_顺序表实现.py
@time: 2021/3/7 21:10
**************************************************
"""


class Stack(object):
    """
    栈
    用 python list （动态顺序表）来实现
    注意，以list尾部为栈顶
    那么push方法时间复杂度为O(1),pop方法的时间复杂度为O(1)

    如果以list头部为栈顶
    那么push方法的 应该为 self.__items.insert(0,item),时间复杂对为 O(N)
    且pop方法 应该为 pop(0)， 时间复杂度为O(N)
    所以这里要注意

    用list的尾部，作为栈顶
    """

    def __init__(self):
        self.__items = []

    def push(self, item):
        """压栈：添加一个新的元素item到栈顶"""
        self.__items.append(item)

    def pop(self):
        """弹出栈顶元素"""
        return self.__items.pop()

    def peek(self):
        """返回栈顶元素"""
```

```python
        if self.__items:  # 判空
            return self.__items[-1]
        else:
            return None

    def is_empty(self):
        """判断栈是否为空"""
        return self.__items == []
        # 或者
        # return not self.__items

    def size(self):
        """返回栈的元素个数"""
        return len(self.__items)


def main():
    stack = Stack()
    stack.push(1)
    stack.push(2)
    stack.push(3)
    stack.push(4)
    print(stack.size())
    print(stack.peek())
    print(stack.pop())
    print(stack.pop())
    print(stack.pop())
    print(stack.pop())


if __name__ == "__main__":
    main()

```

- 栈结构执行过程

Python 2.7

```
1  class Stack:
2      def __init__(self):
3          self.items = []
4
5      def isEmpty(self):
6          return self.items == []
7
8      def push(self, item):
9          self.items.insert(0,item)
10
11     def pop(self):
12         return self.items.pop(0)
13
14     def peek(self):
15         return self.items[0]
16
17     def size(self):
```

Frames Objects

<< First | < Back | Step 1 of 17 | Forward > | Last >>

line that has just executed

next line to execute

# 6 五、队列

- 队列

  **队列（queue）**是只允许在一端进行插入操作，而在另一端进行删除操作的**线性表**。

  **队列**是一种**先进先出的（First In First Out）**的**线性表**，简称FIFO。允许插入的一端为队尾，允许删除的一端为队头。队列不允许在中间部位进行操作！假设队列是q=（a1，a2，……，an），那么a1就是队头元素，而an是队尾元素。这样我们就可以删除时，总是从a1开始，而插入时，总是在队列最后。这也比较符合我们通常生活中的习惯，排在第一个的优先出列，最后来的当然排在队伍最后。

---

# 6.1 队列的实现

- 同栈一样，队列也可以用**顺序表**或者**链表**实现。
- 队列的操作
  - **Queue()** 创建一个空的队列
  - **enqueue(item)** 往队列中添加一个item元素
  - **dequeue()** 从队列头部删除一个元素
  - **is_empty()** 判断一个队列是否为空
  - **size()** 返回队列的大小
- 队列实现代码

```
# -*- coding: utf-8 -*-

"""
**************************************************
@author:   Ying
@software: PyCharm
@file: 11、queue_队列(顺序表实现).py
@time: 2021/3/7 21:44
**************************************************
"""


class Queue(object):
    """
```

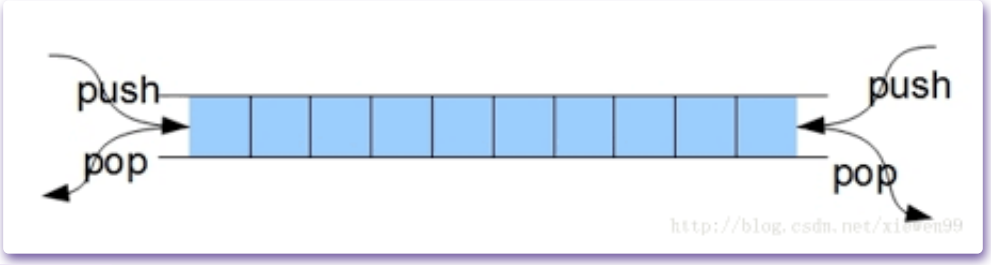
```python
    队列
    用 python list （动态顺序表）
    这里与栈不一样
    因为队列是从一端进，另一端出，所以无论list的头部或者尾部作为队列的尾部（添加端）
    入队列和出队列的时间复杂度总有一个是 O(1) 和 O(N)

    所以到底是选list的头部或者尾部作为队列的尾部i（添加端），要看你经常做的操作到底是出队还是入队
    """

    def __init__(self):
        self.__items = []

    def enqueue(self, item):
        """往队列中添加一个item元素"""
        self.__items.append(item)
        # 下面的是用list 头部最为 队列的头部
        # self.__items.insert(0, item)

    def dequeue(self):
        """从队列头部删除一个元素"""
        return self.__items.pop(0)
        # 下面的是用list 头部最为 队列的头部
        # return self.__items.pop()

    def is_empty(self):
        """判断一个队列是否为空"""
        return self.__items == []

    def size(self):
        """返回队列的大小"""
        return len(self.__items)


def main():
    queue = Queue()
    queue.enqueue(1)
    queue.enqueue(2)
    queue.enqueue(3)
    queue.enqueue(4)
    print(queue.size())
    print(queue.dequeue())
    print(queue.dequeue())
    print(queue.dequeue())
    print(queue.dequeue())


if __name__ == '__main__':
    main()

```

## 6.2 双端队列

- 双端队列 （deque，全名double-ended queue）

  是一种具有队列和栈的性质的数据结构。

  双端队列中的元素可以从两端弹出，其限定插入和删除操作在表的两端进行。双端队列可以在队列任意一端入队和出队。




- 双端队列的操作
  - **Deque()** 创建一个空的双端队列
  - **add_front(item)** 从队头加入一个item元素
  - **add_rear(item)** 从队尾加入一个item元素
  - **remove_front()** 从队头删除一个item元素
  - **remove_rear()** 从队尾删除一个item元素
  - **is_empty()** 判断双端队列是否为空
  - **size()** 返回队列的大小
- 双端队列实现代码

```
# -*- coding: utf-8 -*-

"""
**************************************************
@author:   Ying
@software: PyCharm
@file: 12、deque_双端队列(顺序表实现).py

@time: 2021/3/7 22:22
**************************************************
"""


class Deque(object):
    """
    双端队列
    相当于两个栈的栈底和到一起了
    """

    def __init__(self):
        """利用python list 创建一个双端队列"""
```

```python
        self.__items = []

    def add_front(self, item):
        """从队头加入一个item元素"""
        self.__items.insert(0, item)

    def add_rear(self, item):
        """从队尾加入一个item元素"""
        self.__items.append(item)

    def remove_front(self):
        """从队头删除一个item元素"""
        return self.__items.pop(0)

    def remove_rear(self):
        """从队尾删除一个item元素"""
        return self.__items.pop()

    def is_empty(self):
        """判断双端队列是否为空"""
        return self.__items == []

    def size(self):
        """返回队列的大小"""
        return len(self.__items)


def main():
    deque = Deque()
    deque.add_front(1)
    deque.add_front(2)
    deque.add_rear(3)
    deque.add_rear(4)
    print(deque.size())
    print(deque.remove_front())
    print(deque.remove_front())
    print(deque.remove_rear())
    print(deque.remove_rear())


if __name__ == "__main__":
    main()

```

# 7 六、排序与搜索

- 排序算法

  排序算法（英语：Sorting algorithm）是一种能将一串数据依照特定顺序进行排列的一种算法。

  对之前线性表插入的无序数据进行排序。

- 排序算法的稳定性

  - **稳定性**：稳定排序算法会让原本有相等键值的纪录维持相对次序。也就是如果一个排序算法是稳定的，当有两个相等键值的纪录R和S，且在原本的列表中R出现在S之前，在排序过的列表中R也将会是在S之前。

    当相等的元素是无法分辨的，比如像是整数，稳定性并不是一个问题。然而，假设以下的数对将要以他们的第一个数字来排序。

    ```
    (4, 1)  (3, 1)  (3, 7) (5, 6)
    ```

    在这个状况下，有可能产生两种不同的结果，一个是让相等键值的纪录维持相对的次序，而另外一个则没有：

    ```
    (3, 1)  (3, 7)  (4, 1)  (5, 6)   (维持次序)
    (3, 7)  (3, 1)  (4, 1)  (5, 6)   (次序被改变)
    ```

    **不稳定排序算法可能会在相等的键值中改变纪录的相对次序，但是稳定排序算法从来不会如此。**不稳定排序算法可以被特别地实现为稳定。作这件事情的一个方式是人工扩充键值的比较，如此在其他方面相同键值的两个对象间之比较，（比如上面的比较中加入第二个标准：第二个键值的大小）就会被决定使用在原先数据次序中的条目，当作一个同分决赛。然而，要记住这种次序通常牵涉到额外的空间负担.

---

## 7.1 冒泡排序

- 冒泡排序

  **(Bubble Sort)**是一种简单的排序算法。它重复地遍历要排序的数列，一次比较两个元素，如果他们的顺序错误就把他们交换过来。遍历数列的工作是重复地进行直到没有再需要交换，也就是说该数列已经排序完成。这个算法的名字由来是因为越小的元素会经由交换慢慢“浮”到数列的顶端。

- 冒泡排序算法运作

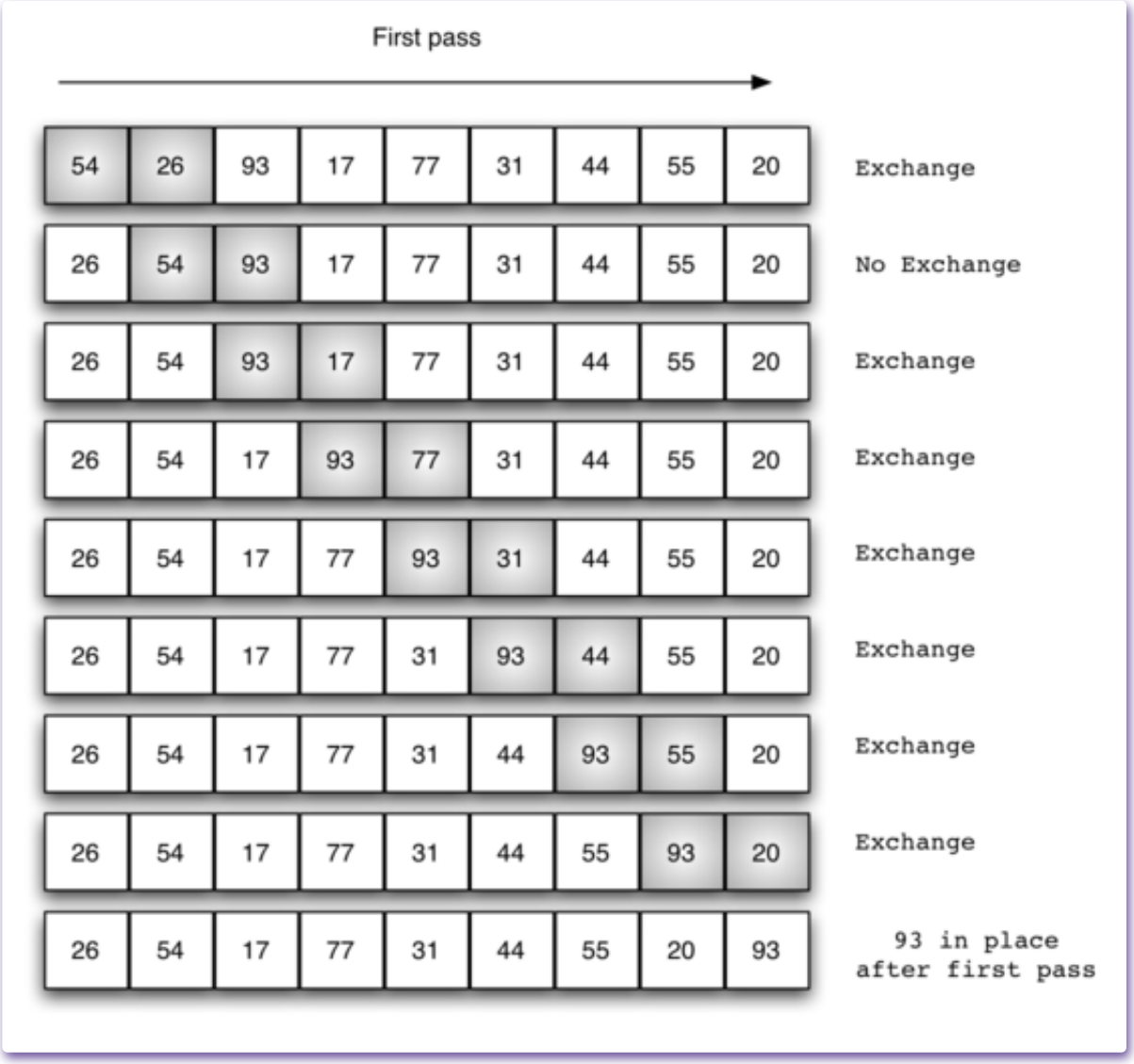
  - 比较相邻的元素。如果第一个比第二个大，即前面的比后面的大（升序），就交换他们两个。

    ```
    if alist[i] > alist[i+1]:
      alist[i],alist[i+1] = alist[i+1] ,alist[i]
    ```

  - 对每一对相邻元素作同样的工作，从开始第一对到结尾的最后一对。这步做完

后，最后的元素会是最大的数。
    - 针对所有的元素重复以上的步骤，除了最后一个。
    - 持续每次对越来越少的元素重复上面的步骤，直到没有任何一对数字需要比较。
- 冒泡排序的分析 （交换过程第一次）

  交换过程图示（第一次）




那么我们需要进行n-1次的冒泡过程，每次对应的比较次数如下所示：

| Pass | Comparisons |
| --- | --- |
| 1 | n-1 |
| 2 | n-2 |
| 3 | n-3 |
| ... | ... |
| n-1 | 1 即 (n-(n-1)) |

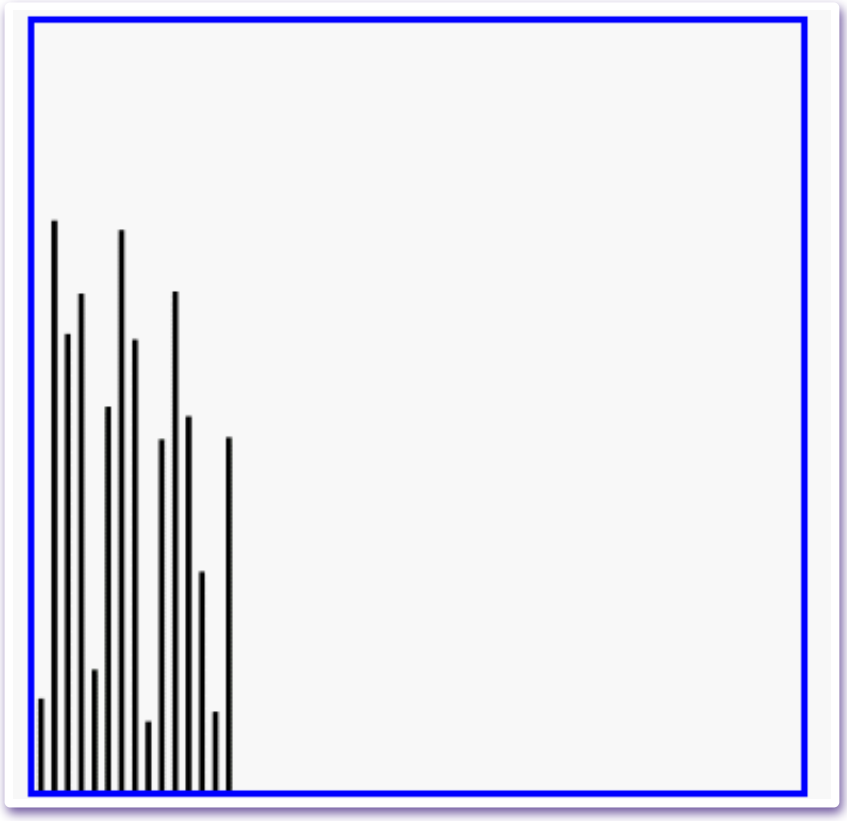
- 冒泡排序代码

注意下面是两种写法

```
# -*- coding: utf-8 -*-

13、冒  泡排序.py
from tqdm import tqdm

def bubble_sort_1(alist):
    """冒泡排序"""
    n = len(alist)
    for j in tqdm(range(0, n - 1)):
        count = 0
        for i in range(n - 1 - j):
            # 从头走到尾
            if alist[i] > alist[i + 1]:
                alist[i], alist[i + 1] = alist[i + 1], alist[i]
                count += 1

    if count == 0: # 优化 如果遍历一遍后发现列表是有序的，那么时间复杂度为O(n)
      return

"""
先写内层循环
第1次 i: 0  ~ n-2   range(0,n-1)   j = 0
第2次 i: 0  ~ n-3   range(0,n-1 - 1)   j = 1
第3次 i: 0  ~ n-4   range(0,n-1 - 2)   j = 2
....
第j次 i: 0  ~ n-2   range(0,n-1 - j)   j

下面的和上面的效果一样
"""


def bubble_sort_2(alist):
    for j in range(len(alist) - 1, 0, -1):
        count = 0
        # j表示每次遍历需要比较的次数，是逐渐减小的
        for i in range(j):
```

```
            if alist[i] > alist[i + 1]:
                alist[i], alist[i + 1] = alist[i + 1], alist[i]
                count += 1

        if count == 0:  # 优化 如果遍历一遍后发现列表是有序的，那么时间复杂度为O(n)
            return


if __name__ == "__main__":
    li = [54, 26, 93, 17, 77, 31, 44, 55, 20]
    bubble_sort_1(li)
    print(li)

# 运算结果
[54, 26, 93, 17, 77, 31, 44, 55, 20]
[17, 20, 26, 31, 44, 54, 55, 77, 93]
100%|██████████| 8/8 [00:00<00:00, 88069.38it/s]
```

- 冒泡排序**时间复杂度**
    - **最优时间复杂度**：$O(n)$（表示遍历一次发现没有任何可以交换的元素，排序结束。）
    - **最坏时间复杂度**：$O(n^2)$
    - **稳定性**：稳定
- **冒泡排序的演示**



# 7.2 选择排序

- **选择排序**

  **选择排序（Selection sort）**是一种简单直观的排序算法。它的工作原理如下。首先在未排序序列中找到最小（大）元素，存放到排序序列的起始位置，然后，再从剩余未排序元素中继续寻找最小（大）元素，然后放到已排序序列的末尾。以此类推，直到所有元素均排序完毕。

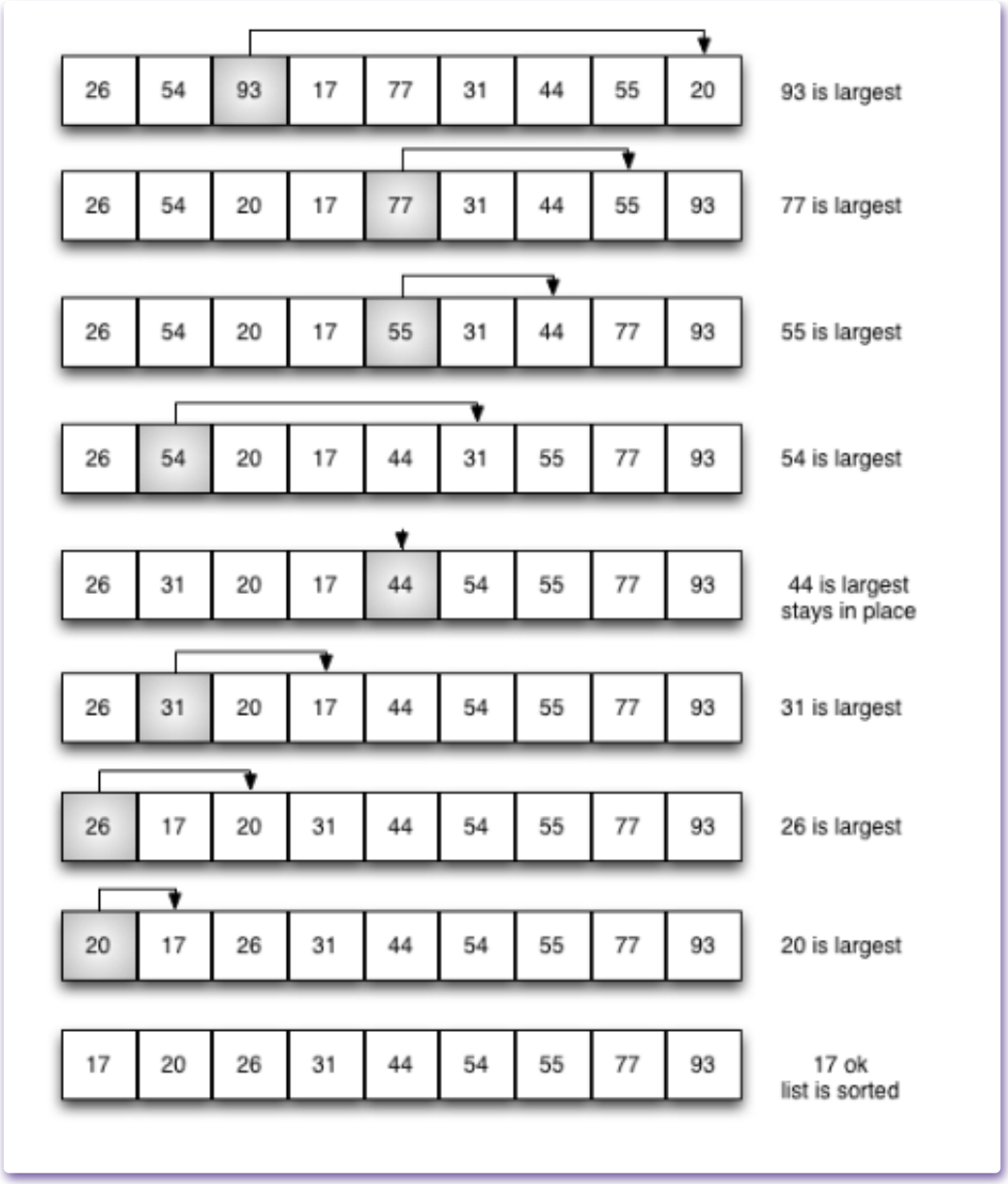
  选择排序的主要优点与数据移动有关。如果某个元素位于正确的最终位置上，则它不会被移动。选择排序每次交换一对元素，它们当中至少有一个将被移到其最终位置上，因此对n个元素的表进行排序总共进行至多n-1次交换。在所有的完全依靠交换去移动元素的排序方法中，选择排序属于非常好的一种。

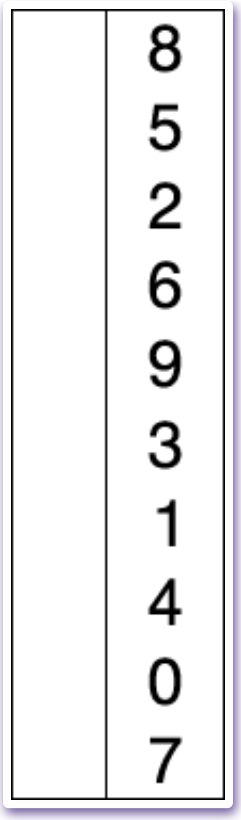
  根据升序，倒序，主要分为以下四种：

  1. 考虑升序，每次选择最小的情况 （最小的放在前面）
  2. 考虑升序，每次选择最大的情况 (最大的放在后面)
  3. 考虑降序，每次选择最小的情况 （最小的放在前面）
  4. 考虑降序，每次选择最大的情况 (最大的放在后面)

- 选择排序分析

  排序过程（考虑升序，每次选择最大的情况）


26 54 93 17 77 31 44 55 20
93 is largest
26 54 20 17 77 31 44 55 93
77 is largest
26 54 20 17 55 31 44 77 93
55 is largest
26 54 20 17 44 31 55 77 93
54 is largest
26 31 20 17 44 54 55 77 93
44 is largest
stays in place
26 31 20 17 44 54 55 77 93
31 is largest
26 17 20 31 44 54 55 77 93
26 is largest
20 17 26 31 44 54 55 77 93
20 is largest
17 20 26 31 44 54 55 77 93
17 ok
list is sorted




红色表示当前最小值，黄色表示已排序序列，蓝色表示当前位置。

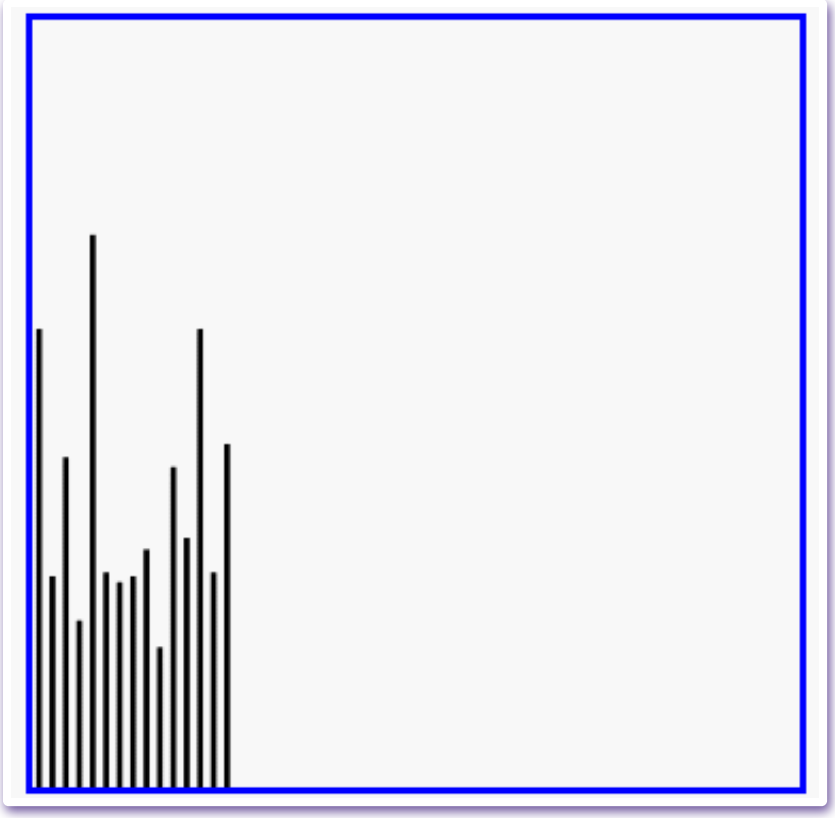
- 选择排序代码

```python
# -*- coding: utf-8 -*-

"""
**************************************************
@author:   Ying
@software: PyCharm
@file: 14、选择排序.py
@time: 2021/3/8 10:18
**************************************************
"""
from tqdm import tqdm

def select_sort(alist):
    """
    选择排序
    1、考虑升序，每次选择最小的情况    (最小的放在前面)
    2、考虑升序，每次选择最大的情况    (最大的放在后面)
    3、考虑降序，每次选择最小的情况    (最小的放在前面)
    4、考虑降序，每次选择最大的情况    (最大的放在后面)

    下面的代码是考虑升序，每次选择最小的情况
    """
    n = len(alist)
    for j in tqdm(range(0, n - 1)):
        min_index = j  # 记录最小值的下标 ，每次迭代开始，认为最小下标为j
        for i in range(j + 1, n):
            if alist[min_index] > alist[i]:
```

```
                    min_index = i
            alist[j], alist[min_index] = alist[min_index], alist[j]


if __name__ == "__main__":
    li = [54, 26, 93, 17, 77, 31, 44, 55, 20]
    print(li)
    select_sort(li)
    print(li)

"""
选择排序没有办法优化
因为必须每个都比较了之后才能知道min_index记录的是否是最小值的下标
所以选择排序的时间复杂度为O(n^2)

选择排序不稳定(考虑升序排序，每次选择最大值放后面的情况)
若
li = [(26,1),1,2,3,16,(26,2),15]
那么选择排序后的结果是
li = [1,2,3,15,16,(26,2),(26,1)]
"""

# 运算结果
[54, 26, 93, 17, 77, 31, 44, 55, 20]
[17, 20, 26, 31, 44, 54, 55, 77, 93]
100%|██████████| 8/8 [00:00<00:00, 87838.83it/s]
```

- 选择排序**时间复杂度**
    - 最优时间复杂度：$O(n^2)$
    - 最坏时间复杂度：$O(n^2)$
    - 稳定性：不稳定（考虑升序每次选择最大的情况）
- 选择排序**演示**

  升序每次选择最大的情况

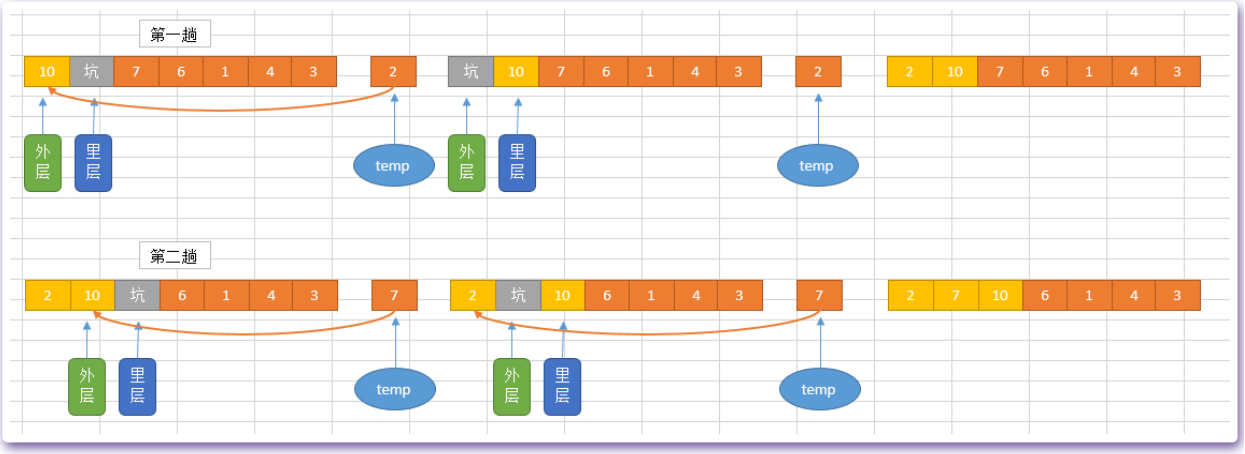


## 7.3 插入排序

- 插入排序

  插入排序（英语：Insertion Sort）是一种简单直观的排序算法。它的工作原理是通过构建有序序列，对于未排序数据，在已排序序列中从后向前扫描，找到相应位置并插入。插入排序在实现上，在从后向前扫描过程中，需要反复把已排序元素逐步向后挪位，为最新元素提供插入空间。

- 插入排序分析




- 插入排序**代码**

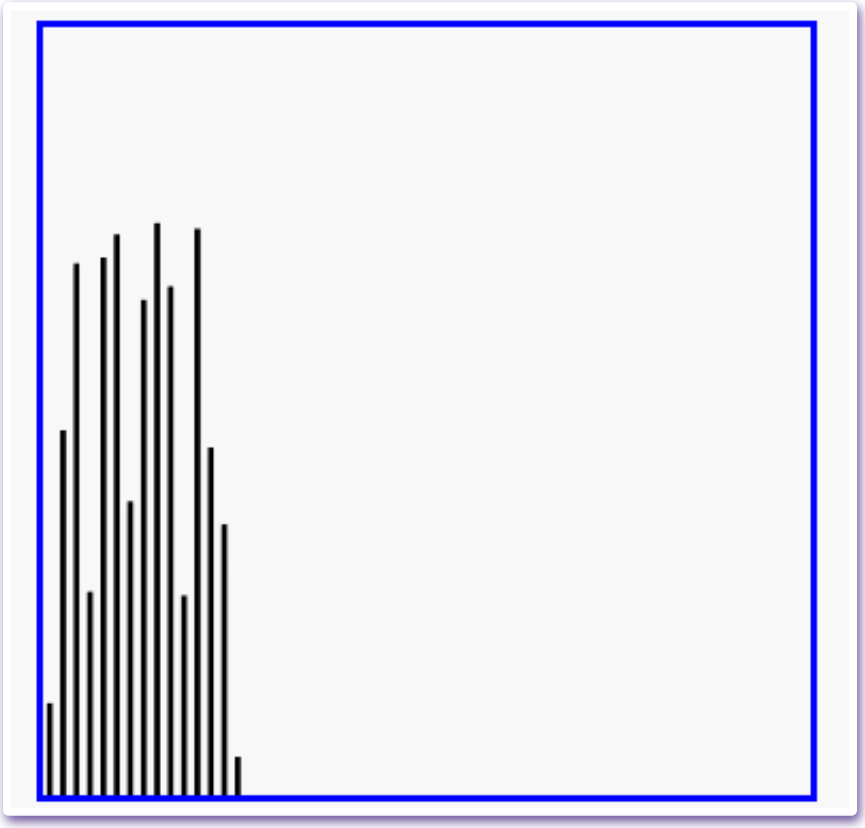
```
# -*- coding: utf-8 -*-
```

```python

"""
**************************************************
@author:   Ying
@software: PyCharm
@file: 15、插入排序.py
@time: 2021/3/8 11:20
**************************************************
"""
from tqdm import tqdm


def insert_sort_1(alist):
    """插入排序"""
    n = len(alist)
    # 从右边的无序序列中取出多少个元素执行这样的过程
    for j in tqdm(range(1, n)):
        # i 代表内层循环起始值
        i = j
        # 执行从右边的无序序列中取出第一个元素，即i位置的元素，然后将其插入到前面的正确的位置中
        while i > 0:
            if alist[i] < alist[i - 1]:
                alist[i], alist[i - 1] = alist[i - 1], alist[i]
                i -= 1
            else:# 对内层循环的优化，使得如果是有序列表的话，插入排序的最优时间复杂度是o(n)
                break


if __name__ == "__main__":
    li = [54, 26, 93, 17, 77, 31, 44, 55, 20]
    print(li)
    insert_sort_1(li)
    print(li)


"""
插入排序
最坏时间复杂度为 O(n^2)
最优时间复杂度为 O(n)

稳定性：稳定
"""

# 运行结果
[54, 26, 93, 17, 77, 31, 44, 55, 20]
[17, 20, 26, 31, 44, 54, 55, 77, 93]
100%|██████████| 8/8 [00:00<00:00, 99864.38it/s]
```

- 插入排序**时间复杂度**

- **最优**时间复杂度：$O(n)$ （升序排列，序列已经处于升序状态）
- **最坏**时间复杂度：$O(n^2)$
- 稳定性：稳定

- 插入排序**演示**



---

## 7.4 希尔排序

- 希尔排序

  (Shell Sort)是插入排序的一种。也称缩小增量排序，是直接插入排序算法的一种更高效的改进版本。希尔排序是非稳定排序算法。该方法因DL．Shell于1959年提出而得名。希尔排序是把记录按下标的一定增量分组，对每组使用直接插入排序算法排序；随着增量逐渐减少，每组包含的关键词越来越多，当增量减至1时，整个文件恰被分成一组，算法便终止。

- 希尔排序过程

  希尔排序的**基本思想**是：将数组列在一个表中并对列**分别进行插入排序**，重复这过程，不过每次用更长的列（步长更长了，列数更少了）来进行。最后整个表就只有一列了。将数组转换至表是为了更好地理解这算法，算法本身还是使用数组进行排序。

  例如，假设有这样一组数$[13, 14, 94, 33, 82, 25, 59, 94, 65, 23, 45, 27, 73, 25, 39, 10]$，如果我们以步长为5开始进行排序，我们可以通过将这列表放在有5列的表中来更好地描述算法，这样他们就应该看起来是这样(竖着的元素是步长组成)：

```
13 14 94 33 82
25 59 94 65 23
45 27 73 25 39
10
```

然后我们对每一列排序：

```
10 14 73 25 23
13 27 94 33 39
25 59 94 65 82
45
```

将上述四行数字，依序接在一起时我们得到：$[10, 14, 73, 25, 23, 13, 27, 94, 33, 39, 25, 59, 94, 65, 82, 45]$。这时10已经移至正确位置了，然后再以3为步长进行排序：

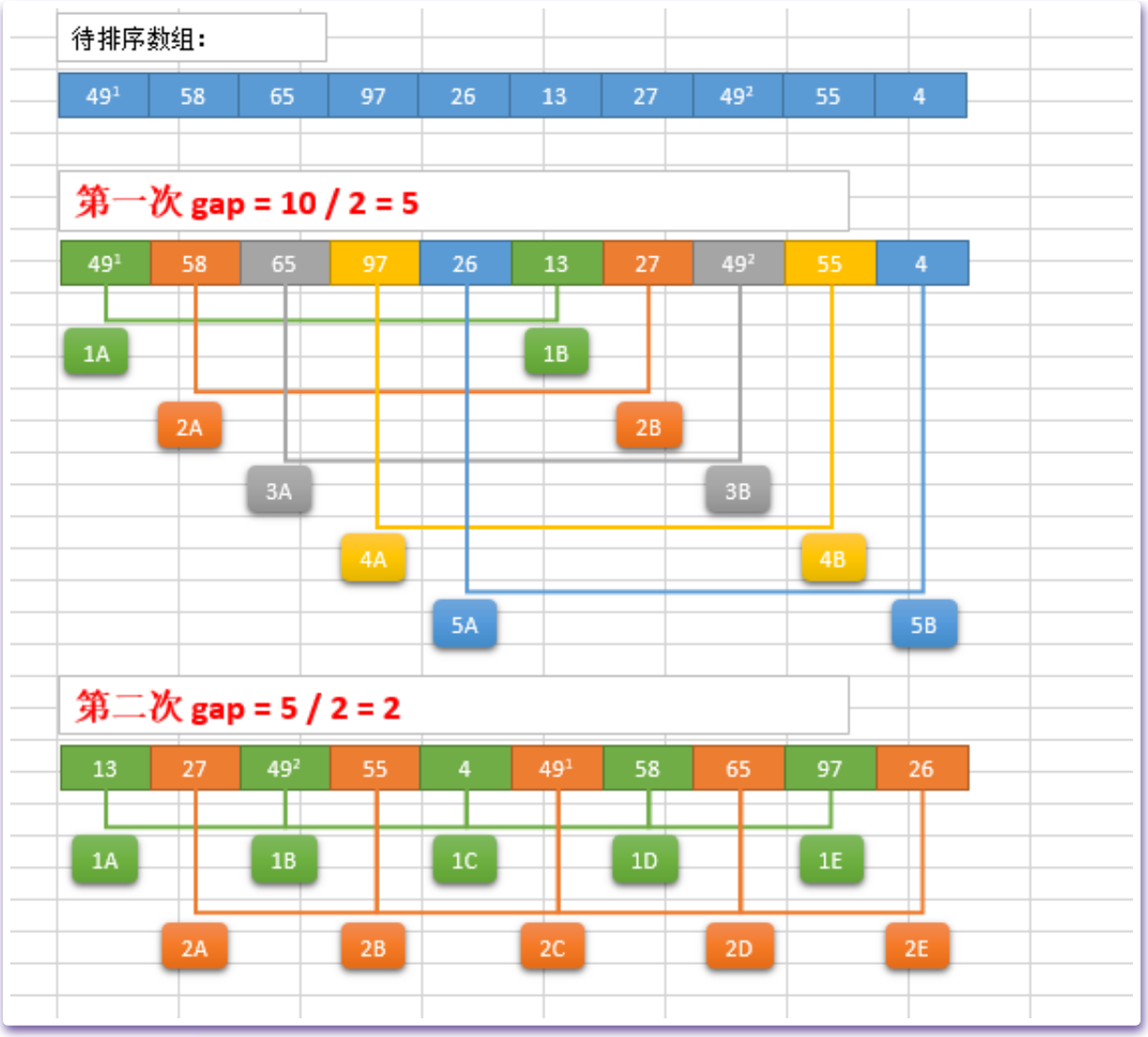
```
10 14 73
25 23 13
27 94 33
39 25 59
94 65 82
45
```

排序之后变为：

```
10 14 13
25 23 33
27 25 59
39 65 73
45 94 82
94
```

最后以1步长进行排序（此时就是简单的插入排序了）

- 希尔排序的分析

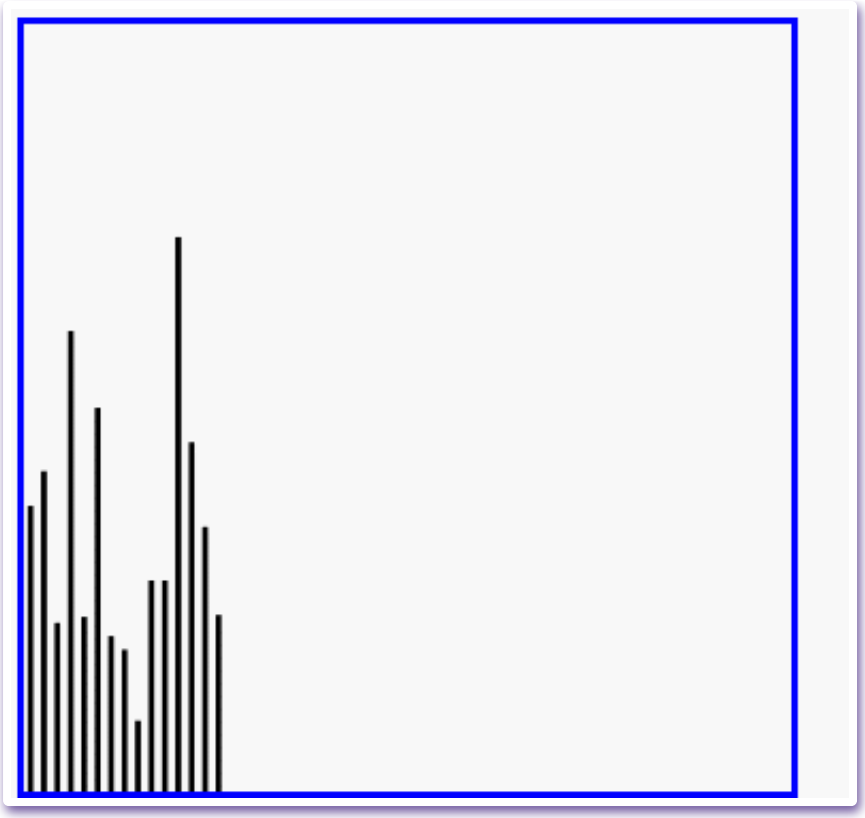



- * *希尔排序代码*

```python
# -*- coding: utf-8 -*-

"""
**************************************************
@author:   Ying
@software: PyCharm
@file: 16、希尔排序.py
@time: 2021/3/8 12:35
**************************************************
"""


def shell_sort(alist):
    """希尔排序"""
    n = len(alist)
    gap = n // 2  # 取整函数
    # gap变化到0之前,插入算法执行的次数
    while gap > 0:
        # 以下是插入算法，与普通的插入算法的区别就是gap步长
        for j in range(gap, n):
            # j = [gap,gap+1,gap+2,...,n-1]
```

```python
            i = j
            while i > 0:
                # if alist[i] < alist[i - gap]:  # 降序
                if alist[i] < alist[i - gap]:  # 升序
                    alist[i], alist[i - gap] = alist[i - gap], alist[i]
                    i -= gap
                else:
                    break
        # 缩短gap步长
        gap //= 2


if __name__ == "__main__":
    li = [54, 26, 93, 17, 77, 31, 44, 55, 20]
    print(li)
    shell_sort(li)
    print(li)

"""
希尔排序
稳定性：不稳定
"""
```

- 希尔排序**时间复杂度**
    - **最优**时间复杂度：根据步长序列的不同而不同
    - **最坏**时间复杂度：O(n2)
    - **稳定性**：不稳定
- 希尔排序演示



## 7.5 快速排序（必须要掌握）

- 快速排序

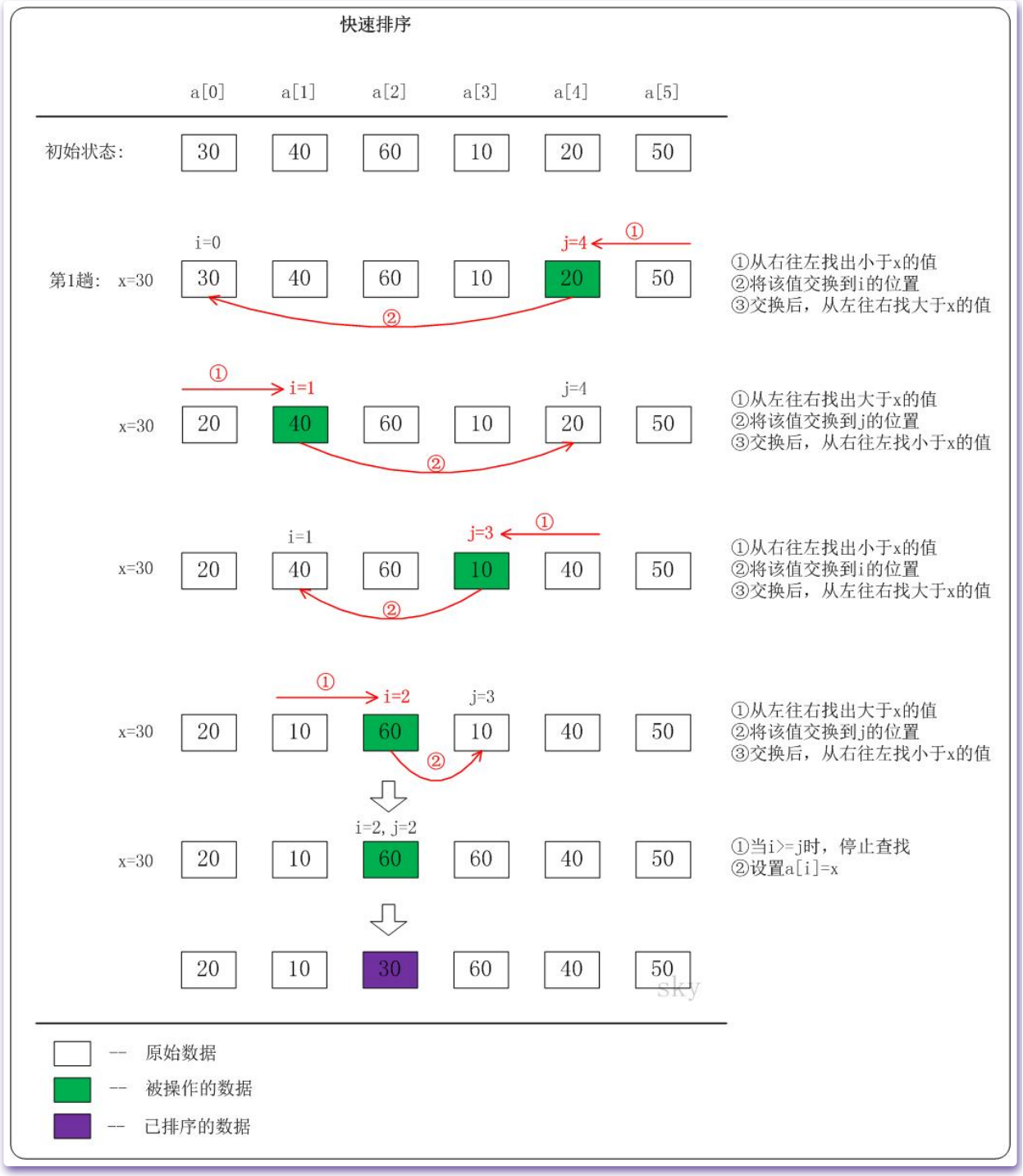
  (英语：Quicksort)，又称划分交换排序（partition-exchange sort），通过一趟排序将要排序的数据分割成独立的两部分，其中一部分的所有数据都比另外一部分的所有数据都要小，然后再按此方法对这两部分数据分别进行快速排序，整个排序过程可以递归进行，以此达到整个数据变成有序序列。

  步骤为：

  1. 从数列中挑出一个元素，称为"基准"（pivot）；
  2. 重新排序数列，所有元素比基准值小的摆放在基准前面，所有元素比基准值大的摆在基准的后面（相同的数可以到任一边）。在这个分区结束之后，该基准就处于数列的中间位置。这个称为分区（partition）操作。
  3. 递归地（recursive）把小于基准值元素的子数列和大于基准值元素的子数列排序。

  递归的最底部情形，是数列的大小是零或一，也就是永远都已经被排序好了。虽然一直递归下去，但是这个算法总会结束，因为在每次的迭代（iteration）中，它至少会把一个元素摆到它最后的位置去。

- 快速排序的分析




- 快速排序代码

```python
# -*- coding: utf-8 -*-

"""
**************************************************
@author:   Ying
@software: PyCharm
@file: 17、快速排序.py
@time: 2021/3/8 15:50
**************************************************
"""


```

```python
def quick_sort(alist, first, last):
    """快速排序"""
    if first >= last:
        return
    mid_value = alist[first]
    low = first
    high = last

    while low < high:
        # 等于的情况只放在一边
        # 如果 low 在 high 的左边，并且 high 位置的元素大于 mid_value high 左移
        while low < high and alist[high] >= mid_value:
            high -= 1
        alist[low] = alist[high]

        # 如果 low 在 high 的左边，并且 low  位置的元素小于 mid_value low  右移
        while low < high and alist[low] < mid_value:
            low += 1
        alist[high] = alist[low]
    # 从循环退出时候，low=high
    alist[low] = mid_value

    # 对low左边的列表执行快速排序
    quick_sort(alist, first, low - 1)

    # 对low右边的列表排序
    quick_sort(alist, low + 1, last)


if __name__ == "__main__":
    li = [54, 26, 93, 17, 77, 31, 44, 55, 20]
    print(li)
    quick_sort(li, 0, len(li) - 1)
    print(li)
```

- 时间复杂度
    - **最优**时间复杂度：$O(n\log n)$
    - **最坏**时间复杂度：$O(n^2)$

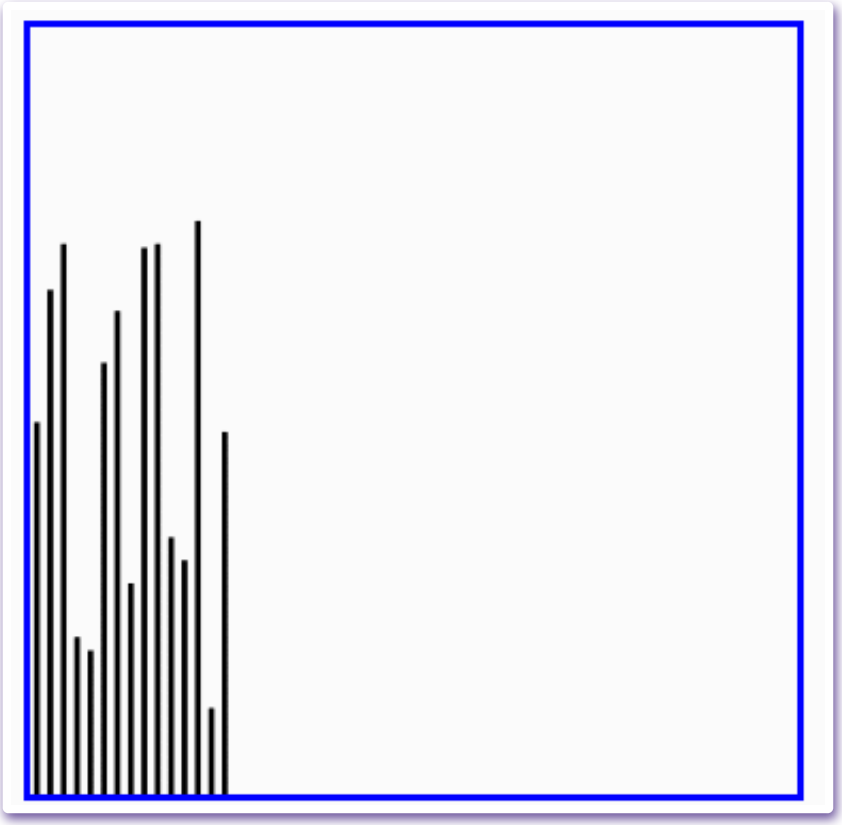
      快速排序更倾向于时间复杂度为$O(n\log n)$
    - **稳定性**：不稳定

从一开始快速排序平均需要花费$O(n\log n)$时间的描述并不明显。但是不难观察到的是分区运算，数组的元素都会在每次循环中走访过一次，使用$O(n)$的时间。在使用结合（concatenation）的版本中，这项运算也是$O(n)$。

在最好的情况，每次我们运行一次分区，我们会把一个数列分为两个几近相等的片段。这个意思就是每次递归调用处理一半大小的数列。因此，在到达大小为一的数列前，我们只要作$\log n$次嵌套的调用。这个意思就是调用树的深度是$O(\log n)$。但是在同一层次结构的

两个程序调用中，不会处理到原来数列的相同部分；因此，程序调用的每一层次结构总共全部仅需要$O(n)$的时间（每个调用有某些共同的额外耗费，但是因为在每一层次结构仅仅只有$O(n)$个调用，这些被归纳在$O(n)$系数中）。结果是这个算法仅需使用$O(n \log n)$时间。

- 快速排序演示



## 7.6 归并排序

- **归并排序**

  归并排序是采用分治法的一个非常典型的应用。归并排序的思想就是先递归分解数组，再合并数组。

  将数组分解最小之后，然后合并两个有序数组，基本思路是比较两个数组的最前面的数，谁小就先取谁，取了后相应的指针就往后移一位。然后再比较，直至一个数组为空，最后把另一个数组的剩余部分复制过来即可。

- 归并排序**分析**

6 5 3 1 8 7 2 4

- 归并排序代码

```python
# -*- coding: utf-8 -*-

"""
**************************************************
@author:   Ying
@software: PyCharm
@file: 18、归并排序.py
@time: 2021/3/8 17:36
**************************************************
"""


def merge_sort(alist):
    """归并排序|"""
    n = len(alist)
    if n <= 1:
        return alist
    mid = n // 2

    # left  代表的是采用归并排序后形成的有序的新的列表
    left_list = merge_sort(alist[:mid])

    # right 代表的是采用归并排序后形成的有序的新的列表
    right_list = merge_sort(alist[mid:])

    # 将两个有序的子序列合并为新的整体
    # merge(left,right)
    left_pointer, right_pointer = 0, 0
    result_list = []

    while left_pointer < len(left_list) and right_pointer < len(right_list):
        if left_list[left_pointer] < right_list[right_pointer]:
            result_list.append(left_list[left_pointer])
            left_pointer += 1
        else:
            result_list.append(right_list[right_pointer])
            right_pointer += 1
    result_list += left_list[left_pointer:]
```

```python
    result_list += right_list[right_pointer:]
    return result_list


if __name__ == "__main__":
    li = [54, 26, 93, 17, 77, 31, 44, 55, 20]
    print(li)
    result_list = merge_sort(li)
    print(result_list)


"""
归并算法需要额外的空间开销
"""
```

## 7.7 常见排序算法效率比较

- 排序算法效率比较

| 排序方法 | 平均情况 | 最好情况 | 最坏情况 | 辅助空间 | 稳定性 |
|---|---|---|---|---|---|
| 冒泡排序 | $O(n^2)$ | $O(n)$ | $O(n^2)$ | $O(1)$ | 稳定 |
| 选择排序 | $O(n^2)$ | $O(n^2)$ | $O(n^2)$ | $O(1)$ | 不稳定 |
| 插入排序 | $O(n^2)$ | $O(n)$ | $O(n^2)$ | $O(1)$ | 稳定 |
| 希尔排序 | $O(n \log n)$~$O(n^2)$ | $O(n^{1.3})$ | $O(n^2)$ | $O(1)$ | 不稳定 |
| 堆排序 | $O(n \log n)$ | $O(n \log n)$ | $O(n \log n)$ | $O(1)$ | 不稳定 |
| 归并排序 | $O(n \log n)$ | $O(n \log n)$ | $O(n \log n)$ | $O(n)$ | 稳定 |
| 快速排序 | $O(n \log n)$ | $O(n \log n)$ | $O(n^2)$ | $O(\log n) \sim O(n)$ | 不稳定 |

## 7.8 搜索

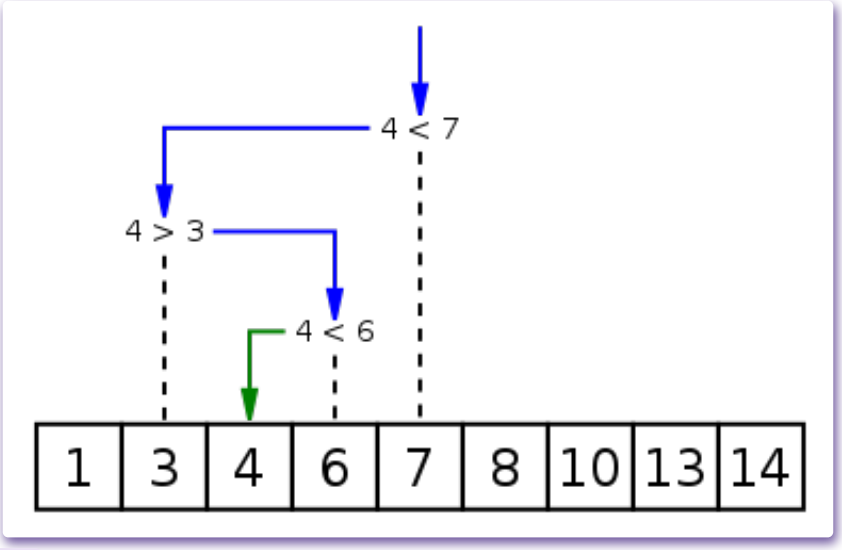
- 搜索

  搜索是在一个项目集合中找到一个特定项目的算法过程。搜索通常的答案是真的或假的，因为该项目是否存在。 搜索的几种常见方法：

  1. 顺序查找
  2. 二分法查找
  3. 二叉树查找
  4. 哈希查找

- 二分法查找

  二分查找又称折半查找，优点是比较次数少，查找速度快，平均性能好；其 缺点 是**要求待查表为有序表，且插入删除困难**。因此，折半查找方法适用于不经常变动而查找频繁的有序列表。首先，假设表中元素是按升序排列，将表中间位置记录的关键字与查找关键字比较，如果两者相等，则查找成功；否则利用中间位置记录将表分成前、后两个子表，如果中间位置记录的关键字大于查找关键字，则进一步查找前一子表，否则进一步查找后一子表。重复以上过程，直到找到满足条件的记录，使查找成功，或直到子表不存在为止，此时查找不成功。




- 二分法查找实现
  - 二分搜索**递归实现**

```python
# -*- coding: utf-8 -*-

"""
**************************************************
@author:   Ying
@software: PyCharm
@file: 19、二分查找.py
@time: 2021/3/8 21:43
**************************************************
"""


def binary_search_1(alist, item):
    """二分查找    递归写法"""
    n = len(alist)
    if n > 0:
        mid = n // 2
        if alist[mid] == item:
            return True
        elif item < alist[mid]:
            return binary_search_1(alist[:mid], item)
        else:
            return binary_search_1(alist[mid + 1:], item)
    return False
```

```python


def main():
    li = [17, 20, 26, 31, 44, 54, 55, 77, 93]
    print(binary_search_1(li, 55))
    print(binary_search_1(li, 100))

if __name__ == "__main__":
    main()

```

- 二分搜索**非递归实现**

```python
def binary_search_2(alist, item):
    """二分查找    非递归查找"""
    n = len(alist)
    first = 0
    last = n - 1
    while first <= last:
        mid = (first + last) // 2
        if alist[mid] == item:
            return True
        elif item < alist[mid]:
            last = mid - 1
        else:
            first = mid + 1
    return False


def main():
    li = [17, 20, 26, 31, 44, 54, 55, 77, 93]
    print(binary_search_2(li, 55))
    print(binary_search_2(li, 100))


if __name__ == "__main__":
    main()

```

- 二分查找**时间复杂度**
  - 最坏时间复杂度：$O(\log n)$
  - 最优时间复杂度：$O(1)$ （第一次就找到了）

---

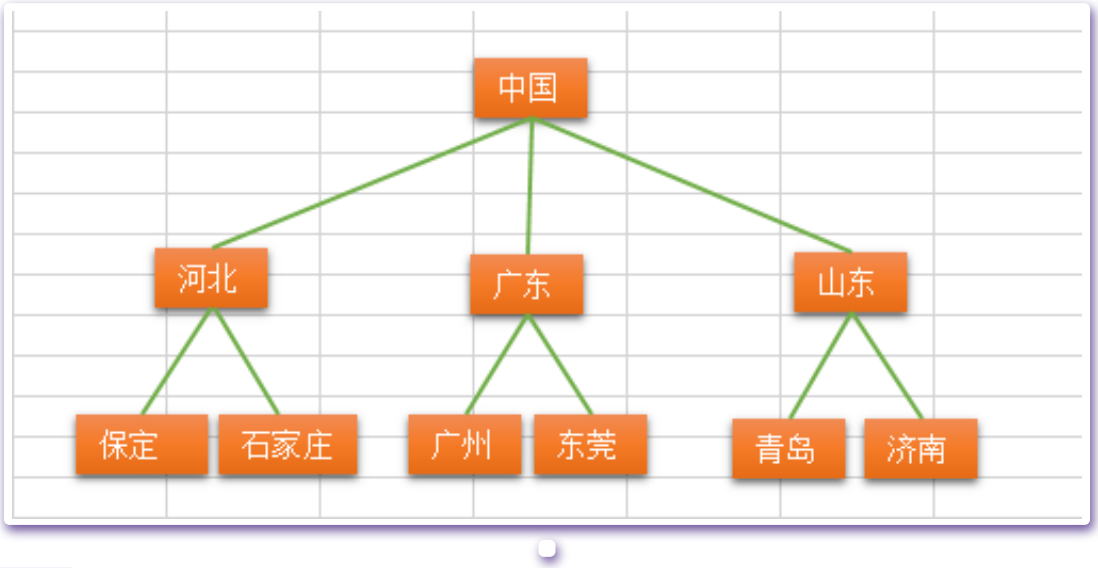
# 8 七、树与树算法

- 树与树算法

  树（英语：tree）是一种抽象数据类型（ADT）或是实作这种抽象数据类型的数据结构，**用来模拟具有树状结构性质的数据集合**。它是由n（n>=1）个有限节点组成一个具有层次关系的集合。把它叫做“树”是因为它看起来像一棵倒挂的树，也就是说它是根朝上，而叶朝下的。它具有以下的特点：

  - 每个节点有零个或多个子节点；
  - 没有父节点的节点称为根节点；
  - 每一个非根节点有且只有一个父节点；
  - 除了根节点外，每个子节点可以分为多个不相交的子树。

  比如：




- 树的术语

  - **节点的度**：一个节点含有的子树的个数称为该节点的度；
  - **树的度**：一棵树中，最大的节点的度称为树的度；
  - **叶节点**或**终端节点**：度为零的节点；
  - **父亲节点**或**父节点**：若一个节点含有子节点，则这个节点称为其子节点的父节点；
  - **孩子节点或子节点**：一个节点含有的子树的根节点称为该节点的子节点；
  - **兄弟节点**：具有相同父节点的节点互称为兄弟节点；
  - 节点的**层次**：从根开始定义起，根为第1层，根的子节点为第2层，以此类推；
  - 树的**高度**或**深度**：树中节点的最大层次；
  - **堂兄弟节点**：父节点在同一层的节点互为堂兄弟；
  - **节点的祖先**：从根到该节点所经分支上的所有节点；
  - **子孙**：以某节点为根的子树中任一节点都称为该节点的子孙；

- **森林**：由m（m>=0）棵互不相交的树的集合称为森林。

- 树的种类
  - **无序树**：树中任意节点的子节点之间没有顺序关系，这种树称为无序树，也称为自由树；
  - **有序树**：树中任意节点的子节点之间有顺序关系，这种树称为有序树；
    - **二叉树**
      1. **完全二叉树**：对于一颗二叉树，假设其深度为d(d>1)。除了第d层外，其它各层的节点数目均已达最大值，且第d层所有节点从左向右连续地紧密排列，这样的二叉树被称为完全二叉树，其中**满二叉树**的定义是所有叶节点都在最底层的完全二叉树;
      2. **平衡二叉树（AVL树）**：当且仅当任何节点的两棵子树的高度差不大于1的二叉树；
      3. **排序二叉树**：二叉查找树（英语：Binary Search Tree），也称二叉搜索树、有序二叉树。即以根节点为中心，所有根节点左边的都比它小，所有根节点右边的都比它大（如下图）。
    - **霍夫曼树**（用于信息编码）：带权路径最短的二叉树称为哈夫曼树或最优二叉树；
    - **B树**：一种对读写操作进行优化的自平衡的二叉查找树，能够保持数据有序，拥有多余两个子树。

- 树的存储与表示

  **顺序存储**：将数据结构存储在固定的数组中，然在遍历速度上有一定的优势，<u>但因所占空间比较大，是非主流二叉树</u>。二叉树通常以链式存储。

  **链式存储**：

  由于对节点的个数无法掌握，常见树的存储表示都转换成二叉树进行处理，子节点个数最多为2。

- 常见的一些树的使用场景
  1. xml，html等，那么编写这些东西的解析器的时候，不可避免用到树；
  2. 路由协议就是使用了树的算法；
  3. mysql数据库索引；
  4. 文件系统的目录结构；
  5. 很多经典的AI算法其实都是树搜索，此外机器学习中的decision tree也是树结构。

---

# 8.1 二叉树

- **二叉树的基本概念**

  二叉树是每个节点最多有两个子树的树结构。通常子树被称作“左子树”（left subtree）和“右子树”（right subtree）

- 二叉树的性质（特性）

  1. 在二叉树的第i层上至多有2^(i-1)个结点（i>0）；
  2. 深度为k的二叉树至多有2^k - 1个结点（k>0）；
  3. 对于任意一棵二叉树，如果其叶结点数为N0，而度数为2的结点总数为N2，则N0=N2+1；
  4. 在二叉树的第i层上至多有2^(i-1)个结点（i>0）；
  5. 对完全二叉树，若从上至下、从左至右编号，则编号为i 的结点，其左孩子编号必为2i，其右孩子编号必为2i＋1；其双亲的编号必为i/2（i＝1 时为根,除外）

- 二叉树种类

  - **完全二叉树**

    若设二叉树的高度为h，除第 h 层外，其它各层 (1～h-1) 的结点数都达到最大个数，第h层有叶子结点，并且叶子结点都是从左到右依次排布，这就是完全二叉树。

  - **满二叉树**

    除了叶结点外每一个结点都有左右子叶且叶子结点都处在最底层的二叉树。

- 二叉树的节点表示以及树的创建

  - 通过使用Node类中定义三个属性，分别为elem本身的值，还有lchild左孩子和rchild右孩子。

```
class Node(object):
    """结点 类"""

    def __init__(self, item):
        self.elem = item
        self.lchild = None
        self.rchild = None

```

  - 树的创建,创建一个树的类，并给一个root根节点，一开始为空，随后添加节点。

```
class Tree(object):
    """二叉树"""

    def __init__(self):
        self.root = None
```

```python

    def add(self, item):
        node = Node(item)
        if self.root is None:
            self.root = node
            return
        queue = [self.root]

        while queue:
            cur_node = queue.pop(0)  # 当前处理的节点
            if cur_node.lchild is None:
                cur_node.lchild = node
                return
            else:
                queue.append(cur_node.lchild)
            if cur_node.rchild is None:
                cur_node.rchild = node
                return
            else:
                queue.append(cur_node.rchild)
```

---

## 8.2 二叉树的遍历（BFS and DFS）

- 二叉树的遍历

  树的遍历是树的一种重要的运算。所谓遍历是指对树中所有结点的信息的访问，即依次对树中每个结点访问一次且仅访问一次，我们把这种对所有节点的访问称为遍历（traversal）。那么树的两种重要的遍历模式是深度优先遍历和广度优先遍历,**深度优先一般用递归，广度优先一般用队列。一般情况下能用递归实现的算法大部分也能用堆栈来实现。**

- 广度优先遍历（BFS:Breadth First Search）
  - 从树的root开始，从上到下从从左到右遍历整个树的节点。

```python
    def breath_travel(self):
        """广度优先遍历"""
        if self.root is None:
            return
        queue = [self.root]
        while queue:
            cur_node = queue.pop(0)
            print(cur_node.elem)
            if cur_node.lchild is not None:
                queue.append(cur_node.lchild)
            if cur_node.rchild is not None:
                queue.append(cur_node.rchild)
```

- 深度优先遍历（DFS:Depth First Search）

  对于一颗二叉树，深度优先搜索(Depth First Search)是沿着树的深度遍历树的节点，尽可能

深的搜索树的分支。
那么深度遍历有重要的三种方法。这三种方式常被用于访问树的节点，它们之间的不同在于访问每个节点的次序不同。这三种遍历分别叫做先序遍历（preorder），中序遍历（inorder）和后序遍历（postorder）。我们来给出它们的详细定义，然后举例看看它们的应用。

- **深度优先遍历：先序遍历**

  在先序遍历中，我们先访问根节点，然后递归使用先序遍历访问左子树，再递归使用先序遍历访问右子树
  **根节点->左子树->右子树**

```
    def preorder(self, node):
        """深度优先遍历 先序遍历（根 左 右） 递归实现"""
        if node is None:
            return
        print(node.elem, end=' ')  # 根
        self.preorder(node.lchild)  # 左
        self.preorder(node.rchild)  # 右
```

- **深度优先遍历：中序遍历**

  在中序遍历中，我们递归使用中序遍历访问左子树，然后访问根节点，最后再递归使用中序遍历访问右子树
  **左子树->根节点->右子树**

```
    def inorder(self, node):
        """深度优先遍历 中序遍历（左 根 右）递归实现"""
        if node is None:
            return
        self.inorder(node.lchild)  # 左
        print(node.elem, end=' ')  # 根
        self.inorder(node.rchild)  # 右
```

- **深度优先遍历：后序遍历**

  在后序遍历中，我们先递归使用后序遍历访问左子树和右子树，最后访问根节点
  **左子树->右子树->根节点**

```
    def postorder(self, node):
        """深度优先遍历 后序遍历（左 右 根）递归实现"""
        if node is None:
            return
        self.postorder(node.lchild)  # 左
        self.postorder(node.rchild)  # 右
        print(node.elem, end=' ')  # 根
```

- 深度优先遍历和广度优先遍历的顺序

  下图是个完全二叉树

- 给出一棵树，如何写出三种遍历的顺序
    1. 广度优先：从上到下，从左到右
    2. 深度优先先序：根、左、右
    3. 深度优先中序：左、根、右
    4. 深度优先后续：左、右、根

    结果：

    1. 广度优先：A-B-F-C-G-D-E-H
    2. 先序：A-B-C-D-E-F-G-H
    3. 中序：D-C--E-B-A-F-H-G
    4. 后续：D-E-C-B-H-G-F-A
- 哪两种遍历方式能够唯一的确定一颗树？？？

  先、中、后序，中序一定要给，其他两个任给一个就可以画出树和写出第三种
- 二叉树完整代码

```python
# -*- coding: utf-8 -*-

"""
**************************************************
@author:   Ying
@software: PyCharm
@file: 20、二叉树.py
@time: 2021/3/11 12:30
**************************************************
"""


class Node(object):
    """节点"""

    def __init__(self, item):
        self.elem = item
        self.lchild = None
        self.rchild = None


class Tree(object):
    """二叉树"""

    def __init__(self):
        self.root = None

```

```python
    def add(self, item):
        node = Node(item)
        if self.root is None:
            self.root = node
            return
        queue = [self.root]

        while queue:
            cur_node = queue.pop(0)  # 当前处理的节点
            if cur_node.lchild is None:
                cur_node.lchild = node
                return
            else:
                queue.append(cur_node.lchild)
            if cur_node.rchild is None:
                cur_node.rchild = node
                return
            else:
                queue.append(cur_node.rchild)

    def breath_travel(self):
        """广度优先遍历"""
        if self.root is None:
            return
        queue = [self.root]
        while queue:
            cur_node = queue.pop(0)
            print(cur_node.elem)
            if cur_node.lchild is not None:
                queue.append(cur_node.lchild)
            if cur_node.rchild is not None:
                queue.append(cur_node.rchild)

    def preorder(self, node):
        """深度优先遍历 先序遍历（根 左 右）递归实现"""
        if node is None:
            return
        print(node.elem, end=' ')  # 根
        self.preorder(node.lchild)  # 左
        self.preorder(node.rchild)  # 右

    def inorder(self, node):
        """深度优先遍历 中序遍历（左 根 右）递归实现"""
        if node is None:
            return
        self.inorder(node.lchild)  # 左
        print(node.elem, end=' ')  # 根
        self.inorder(node.rchild)  # 右

    def postorder(self, node):
        """深度优先遍历 后序遍历（左 右 根）递归实现"""
        if node is None:
```

```
            return
        self.postorder(node.lchild)  # 左
        self.postorder(node.rchild)  # 右
        print(node.elem, end=' ')  # 根


def main():
    tree = Tree()
    tree.add(0)
    tree.add(1)
    tree.add(2)
    tree.add(3)
    tree.add(4)
    tree.add(5)
    tree.add(6)
    tree.add(7)
    tree.add(8)
    tree.add(9)
    # tree.breath_travel()
    print(" ")
    tree.preorder(tree.root)  # 这的参数是告诉它哪里是跟节点
    print(' ')
    tree.inorder(tree.root)  # 这的参数是告诉它哪里是跟节点
    print(' ')
    tree.postorder(tree.root)  # 这的参数是告诉它哪里是跟节点


if __name__ == "__main__":
    main()
```